

Selective Amnesia: On Efficient, High-Fidelity and Blind Suppression of Backdoor Effects in Trojaned Machine Learning Models

Rui Zhu*, Di Tang*, Siyuan Tang, XiaoFeng Wang, Haixu Tang
Indiana University Bloomington
{zhu11, tangd, tangsi, xw7, hatang}@iu.edu

Abstract—The extensive applications of deep neural network (DNN) and its increasingly complicated architecture and supply chain make the risk of backdoor attacks more realistic than ever. In such an attack, the adversary either poisons the training data of a DNN model or manipulates its training process to stealthily inject a covert backdoor task, alongside the primary task, so as to strategically misclassify inputs carrying a trigger. Defending against such an attack, particularly removing the backdoor effect from an infected model, is known to be hard. For this purpose, prior research either requires a recovered trigger, which is hard to come by, or attempts to fine-tune a model on its primary task, which becomes less effective when the clean data is scarce. In this paper, we present a simple yet surprisingly effective technique to induce “selective amnesia” on a backdoored model. Our approach, called SEAM, has been inspired by the problem of *catastrophic forgetting* (CF), a long standing issue in continual learning. Our idea is to retrain a given DNN model on randomly labeled clean data, to induce a CF on the model, leading to a sudden forget on both primary and backdoor tasks; then we recover the primary task by retraining the randomized model on correctly labeled clean data. We analyzed SEAM by modeling the unlearning process as continual learning and further approximating a DNN using Neural Tangent Kernel for measuring CF. Our analysis shows that our random-labeling approach actually maximizes the CF on an unknown backdoor in the absence of triggered inputs, and also preserves some feature extraction in the network to enable a fast revival of the primary task. We further evaluated SEAM on both image processing and Natural Language Processing tasks, under both data contamination and training manipulation attacks, over thousands of models either trained on popular image datasets or provided by the TrojAI competition. Our experiments show that SEAM vastly outperforms the state-of-the-art unlearning techniques, achieving a high *Fidelity* (measuring the gap between the accuracy of the primary task and that of the backdoor) efficiently (e.g., about 30 times faster than training a model from scratch on the MNIST dataset), with only a small amount of clean data (e.g., with a size of just 0.1% of training data for TrojAI models).

1. Introduction

With the wide applications of deep neural networks (DNN), e.g., in image classification [56], natural language

processing (NLP) [43], malware detection [74], etc., come increased security risks, due to the complexity of DNN models and their training pipelines, which open new attack surfaces. These models are known to be vulnerable to various attacks, such as adversarial learning [29] and backdoor injection [31]. Particularly, in a backdoor attack, the adversary either contaminates the training data for a model or manipulates its training process [17], so as to embed a Trojan backdoor into the model; as a result, the model may apparently fulfill its task as anticipated but actually responds to the presence of a predefined pattern, called a *trigger*, by misclassifying the instance carrying the trigger to a wrong label. For example, a backdoored face recognition model for biometric authentication could always identify anyone wearing a unique pair of glasses as a privileged user of a critical system [69].

Challenges in backdoor defense. Effective control of such a backdoor risk is hard. The most explored avenue is detection, which often relies on recovering triggers from DNN models [66] [81] or identifying their anomaly in the presence of noise [26], inputs carrying triggers [5], and others [41]. With some moderate success, backdoor detection is fundamentally challenging: the complexity of the models and the diversity of triggers (size, form, location, similarity to legitimate features, etc.) often render existing techniques (e.g., trigger recovery [66]) futile; even a successful detection often comes with a significant performance overhead, making these approaches less scalable [41]. As a prominent example, a prior study shows that a source-specific backdoor (a trigger only causing misclassification of the images from a certain class) has defeated all existing detection solutions, except the one requiring triggered images (less likely to get in practice) [59].

Following detection is removal of a backdoor from an infected DNN model, which is done through *unlearning*. Specifically, if the trigger has been recovered, one can retrain the model on correctly labeled inputs carrying the trigger to remove the effect of the backdoor [66]. This approach, however, is contingent upon trigger recovery, which is hard in general as mentioned earlier. An alternative is what we call *blind* unlearning, a technique that works on a DNN model regardless of whether it contains a backdoor, in an attempt to weaken or even eliminate the backdoor effect when it does exist. Its efficacy can be measured using *Fi-*

* The first two authors contributed equally to this work.

fidelity (Section 5), a metric we propose to determine the gap between the model’s accuracy for its primary task (*ACC*) and that for its backdoor task (called the *attack success rate* or *ASR*). A high *Fidelity* indicates that the unlearning technique largely preserves the desired classification capability of the model while suppressing its unwanted backdoor effect.

However, achieving the high *Fidelity* in blind unlearning is nontrivial. All existing approaches rely on fine-tuning of a DNN model over a set of clean data, for the purpose of reinforcing the model’s functionality for solving the primary task, thereby implicitly weakening its capability to handle its covert backdoor task (e.g., Fine-pruning [39] and Neural Attention Distillation [38]). The problem is that a backdoored model with a well trained primary task often has little room for fine-tuning (small loss with little impact on its weights). So the effective of this approach can be limited, as acknowledged by the prior research [40], particularly when the clean datasets for fine-tuning are small (below 10%).

Selective amnesia. Ideally, blind unlearning should lead to a “selective amnesia” for the target model, causing it to remove the memory about the hidden backdoor task while keeping that for solving the primary classification task. We believe that this cannot be effectively achieved by the existing approaches, fundamentally due to their lack of means to explicitly forget the unknown backdoor. In the meantime, we found that there exists a surprisingly simple yet effective solution, through inducing a *catastrophic forgetting* (CF) [33] on a DNN model and then recovering its desired functionality using a task similar to its overt primary task. More specifically, given a model, our approach first retrains it on a set of data with random labels to cause a CF, through which the model forgets both its primary classification task and its hidden backdoor task; then we utilize a small set of clean data to train the model on the primary task, leading to the recovery of the task without revival of the backdoor. This approach, which we call SEAM (selective amnesia), turns out to be highly effective: on MNIST, GTSRB and CIFAR10 datasets, the backdoored models processed by SEAM achieve a high *Fidelity* when using a clean dataset with a size of 0.1% of training data for forgetting and 10% for recovery; on the infected models for the TrojAI competition, a clean recovery set, just 0.1% of the training data size, is found to be enough to completely suppress those models’ backdoor effects, attaining a high *Fidelity*. The experimental findings demonstrate that SEAM can nearly fully preserve the model’s primary functionality and also almost completely remove the backdoor effect.

To understand why this simple approach works so effectively, we model the backdoor attack as a problem of multi-task learning to analyze the relations between the primary and the covert backdoor tasks and further utilize Neural Tangent Kernel to approximate a backdoored model and measure the CF incurred by a sequence of tasks (*forgetting* and *recovery*). Our analysis shows that our random-labeling task is *optimal* for *forgetting* a hidden backdoor on a given fixed dataset (e.g., a small subset of clean data). Further, under the CF induced by the function, we show that a

recovery process will selectively revive the primary task by training the model on a similar task (even with a much smaller training dataset).

We further evaluated SEAM on DNN models with various architectures (ShuffleNet, VGG, ResNet) for different image recognition and NLP tasks on popular datasets (MNIST, CIFAR10, GTSRB, Imagenet, TrojAI datasets, etc.), under different types of backdoor attacks (Reflection [45] and TrojanNet [61]). In all these tests, SEAM achieved a very high *Fidelity*, nearly fully restoring the original model’s *ACC* and completely removing its backdoor effect, often within a few minutes. Also we ran SEAM against the state-of-the-art unlearning techniques, including Neural Cleanse (NC) [66], Fine-Pruning [39] and Neural Attention Distillation (NAD) [38], and demonstrated that our approach vastly outperforms these solutions: particularly, given only 0.1% of clean training data, SEAM reported a *Fidelity* around 90% in less than 1 minute, while other approaches took around an hour to get far lower results (50% to a bit above 70%). We also analyzed the robustness of our technique against various evasion attempts.

Contributions. Our contributions are outlined as follows:

- *Novel backdoor defense.* We present a new blind unlearning technique that for the first time, utilizes catastrophic forgetting to achieve efficient and *high-Fidelity* removal of Trojan backdoors, in the absence of trigger information.
- *Theoretical understandings.* We model the backdoor attack as a multi-task learning problem and further leverage Neural Tangent Kernel (NTK) to measure CF and the similarity between the overt primary task and the covert backdoor task. Our analysis proves the optimality of our forgetting task (random labeling) and also helps better understand the limitations of other unlearning techniques.
- *Extensive experiments.* We report an extensive evaluation of our new technique, on both image recognition and NLP tasks, under various types of backdoor attacks, and also in comparison with state-of-the-art solutions. Our evaluation shows that the new approach, though simple, is highly effective, vastly outperforming existing techniques and fully suppressing backdoor effects even in the presence of a small amount of clean data.

2. Background

2.1. Backdoor

A backdoor attack aims to induce strategic misclassification in a DNN model through training data poisoning [6], [11], [65] or training process manipulation [16], [62]. The risk of a backdoor attack can be mitigated through detection: discovering triggers using SGD [66], leveraging images carrying triggers [81] and others [72], as mentioned earlier. A prominent example is Neural Cleanse (NC) [66], which searches for the pattern with the anomalously small norm that causes any image with the pattern to be classified into a target label. Once discovered, the pattern can be used to unlearn the model’s backdoor, by training it on the correctly labeled inputs carrying the pattern. Unlike NC, which depends on trigger discovery to unlearn a backdoor

from a DNN model, Fine-pruning [39] and Neural Attention Distillation [38] are designed to directly remove a backdoor from a Trojaned model, without using any information about the trigger. More specifically, Fine-pruning prunes less informative neurons and then finetunes the model [39], in an attempt to directly erase the backdoor effect; NAD first finetunes a given model to create a *teacher* model, which is combined with the original model (the student) through distillation to unlearn a hidden backdoor [38].

2.2. Multi-Task Learning and Continual Learning

In multi-task learning (MTL), several learning tasks are solved jointly at the same time, which could outperform the training alone on individual tasks [67], [79], [80]. More specifically, consider several supervised learning tasks in MTL $\tau_t, t \in [T]$ where $\tau_t = \{x_{t_i}, y_{t_i}\}_{i=1}^{n_t}$, with $T \in \mathbb{N}^+$ the set of positive integers. Suppose $\mathcal{X}$ is a feature space of interest, $\mathcal{X} \subseteq \mathbb{R}^p$, and $\mathcal{Y}$ is the space of labels, $\mathcal{Y} \subseteq \mathbb{N}$, then $X_t \subseteq \mathbb{R}^{n_t \times p}$ (p is the feature dimension) represents the dataset of the task τ_t and $x_{t_i}^t, i = 1, \dots, n_t \in \mathcal{X}$ is a sample with its corresponding label $y_{t_i}^t \in \mathcal{Y}$. The goal of MTL is to learn a function: $f_\omega : \mathcal{X} \times \mathcal{T} \rightarrow \mathcal{Y}$ with $\omega \in \mathbb{R}^q$ being the parameters that fit the prediction as accurate as possible. As a special case of MTL, Continual Learning (CL) [47], [73] solves a stream of supervised learning tasks $\tau_1, \tau_2, \dots, \tau_T$. The goal is to train a predictor with the best accuracy on each task assuming that the training data for previous tasks are no longer available after the tasks are accomplished. In our paper, we utilize the continual learning theory to model the unlearning process used by SEAM. Specifically, we consider the backdoor unlearning as an independent process after the given model (benign or backdoored) has already been trained.

2.3. Neural Tangent Kernel

Neural network is often considered to be a black box model since it may not offer much insight into the function $f(\cdot)$ it approximates. Recently, neural tangent kernel (NTK) has been proposed to provide a set of theoretical tools for understanding neural networks. In the NTK theory, a neural network is approximated by a kernel machine, in which the kernel function is composed of the gradients of neural network parameters with regard to the training data. More specifically, NTK utilizes the Taylor expansion of the network function with respect to the weights around its initialization for the approximation:

$$f(x, w) \approx f(x, w_0) + \nabla_w f(x, w_0)^T (w - w_0),$$

where w_0 is the neural network's initial weights, and $\nabla_w f(x, w_0)$ is what so called "NTK" and can be represented in the feature map form $\phi(x) = \nabla_w f(x, w_0)$ in which ϕ is the feature map in the kernel (NTK) space. Note that, once we choose to expand the initial weights w_0 , the feature map ϕ is fixed. This means that the NTK approximation in the following training procedure utilizes the same feature space, but different training data will optimize each feature's weight. In practice, it is not necessary to perform the expansion around the initial weights w_0 . In fact, the Taylor expansion can have better precision if it is performed

around the weights in a later state w_z when we approximate the neural network after the state z . For more details of the NTK theory, we refer to the prior research [30], and a recent implementation of the NTK framework for a convolutional neural network [3].

2.4. Threat Model

Defender's goal. Removing the backdoor injected into the target model and keeping the accuracy of the backdoor removed model be close to what of the original model.

Defender's capabilities. We assume that the defender has access to the backdoored model and a small set of clean data (with no trigger-carrying samples) but does not know information about the trigger. We believe that the availability of the clean data is realistic, given the fact that the user of an ML model today tend to possess a set of clean testing data for evaluating the model's functionality. For example, Paper With Code [1] provides pre-trained ML models, whose qualities can be evaluated by the user on the public benchmarks released by trusted parties [71]; these benchmarks can serve as the clean dataset. As another example, image classification models tend to be evaluated using ImageNet [9], a public dataset with integrity protection (md5 fingerprints); so even when a model itself is contaminated during its training process, still we can utilize the ImageNet data, which is supposed to be clean, to unlearn its backdoor. We further assume that this small set of the clean data has the same distribution as that of the sample space the primary task is supposed to work in. We make this "same distribution" assumption since the success of recovery relies on the similarity between the distribution of the samples and that of the clean dataset for recovery. This assumption is also required for the training data on which any machine learning model is being built.

Adversary goal. The adversary intends to inject a backdoor into the target model through either training data poisoning or training process manipulation.

Adversarial capabilities. We assume a white-box adversary who can access both data and model parameters during training, but does not have influence on the way the model is modified by its user after the training is complete and the model is released. Particularly, we assume that the adversary can inject a backdoor into the victim model, but cannot interfere with SEAM's unlearning operations on the model. This assumption is reasonable, since when a backdoored model (compromised during its training process) is given to the user, it tends to be outside the adversary's control: otherwise, the adversary can easily tamper with the answer sent back to the user even in the absence of the backdoor, rendering the backdoor attack less meaningful. Also the adversary's capability to pollute the data for the recovery operation or model fine-tuning can be constrained in various real-world scenarios, as the examples described above.

3. Our Method: SEAM

3.1. Motivation

Our research shows that fine-tuning a Trojaned model cannot guarantee removal of its backdoor, particularly when

the clean dataset for fine-tuning is small ($<10\%$ of the model's original training data). This has been acknowledged in the prior research [39], even with its effort to prune suspicious neurons to improve the efficacy of fine-tuning.

Fundamentally, we believe that fine-tuning is limited in its potential to suppress the backdoor effect in general: consider a well-trained but backdoored model, with its ACC close to 1; fine-tuning the model on the clean dataset under its primary (overt) task will have little impact on its weights, given that its loss is already small, and therefore will not significantly interfere with its covert backdoor task.

A natural solution here is to explicitly forget the backdoor information without undermining the model's capability to solve its primary task. This purpose can be served, effectively, by the idea of *Catastrophic Forgetting* (CF), as discovered in our research. CF has long been considered to be a problem for artificial neural network (NN) [34], causing an NN to completely and abruptly lose memory of the previously learnt task when learning the information about a new task. The problem is known to be a major barrier to continual learning [24], since the weights in an NN for storing the knowledge of one task will be changed to meet the requirements of the subsequently learnt task. This long-standing trouble, however, was leveraged in our research to enhance the trustworthiness of a DNN. More specifically, prior research shows that a CF can be induced by a new task with similar input features as the old task but different classification outputs [12]. This property was utilized by us to build a novel pipeline in which a forgetting task is first run to cause the maximum CF (Section 4.3), and then a recovery task is performed to revive the primary task without wakening the backdoor. In this way, we can achieve a selective amnesia on an infected DNN to remove its backdoor effect.

3.2. The SEAM Pipeline

At the center of our blind unlearning pipeline is the forgetting task for inducing a CF on a given DNN model. The task is meant to achieve the following goals: 1) ensuring a large CF and 2) helping a quick and selective recovery from the CF. For 1), we need to largely preserve input feature extraction but incur significant impacts on classification, so as to interfere with the original tasks, including the backdoor task; for 2), we hope that the changes to the weights of the DNN, as caused by the interference, can be easily and selectively reversed, so the primary task can be recovered. To this end, we designed a *random-labeling* task that assigns a random class label to each output of the model. This simple approach utilizes the features the original model discovers on the input and intermediate layers but causes a large loss that leads to significant changes to the weights of the layers close to the output. Further, such changes can be done with a few rounds of updates on the weights through stochastic gradient descent (SGD), so the impacts on the primary task can be quickly reversed.

Algorithm. Based upon this simple forgetting task, we developed the SEAM pipeline, as illustrated in Algorithm 1. SEAM has two steps: *forgetting* and *recovery*, and takes the

following inputs: an NN $f(\cdot)$, a labeled *forgetting* dataset $\mathcal{D}_{for}$ for the forgetting task, the number of epochs N_{for} for running the forgetting task, the accuracy threshold Acc_{for} for an early stop at the forgetting step, a *recovery* dataset $\mathcal{D}_{rec}$ for recovering the primary task, its training epochs N_{rec} , and accuracy threshold Acc_{rec} for an early stop. In Algorithm 1, *Line 1-9* describe the *forgetting* step: starting from the original model $f(\cdot)$, we randomly re-label each sample in $\mathcal{D}_{for}$ with a class different from its desirable class (randomly wrong class), aiming to build a randomly labeled training dataset $\bar{\mathcal{D}}_{for}$ for each epoch to train the model on $\bar{\mathcal{D}}_{for}$; after N_{for} epochs, the resulting model $\bar{f}$ is expected to classify any given input sample to each label with a similar probability. *Line 10-17* show the *recovery* step: we re-train the model $\bar{f}$ on the dataset $\mathcal{D}_{rec}$ for N_{rec} epochs to revive the primary task.

Algorithm 1 SEAM

Input: $f(\cdot), \mathcal{D}_{for}, N_{for}, Acc_{for}, \mathcal{D}_{rec}, N_{rec}, Acc_{rec}$

Output: $\bar{f}(\cdot)$

```

1:  $\bar{f} \leftarrow f$ 
2: for epoch in  $\text{range}(N_{for})$  do
3:    $\bar{\mathcal{D}}_{for} \leftarrow \text{Randomly wrong label } \mathcal{D}_{for}$ 
4:    $\bar{f} \leftarrow \text{Train } \bar{f} \text{ on } \bar{\mathcal{D}}_{for}$ 
5:    $Acc \leftarrow \text{Test } \bar{f} \text{ on } \bar{\mathcal{D}}_{for}$ 
6:   if  $Acc < Acc_{for}$  then
7:     Break
8:   end if
9: end for
10:  $\bar{f} \leftarrow \bar{f}$ 
11: for epoch in  $\text{range}(N_{rec})$  do
12:    $f \leftarrow \text{Train } \bar{f} \text{ on } \mathcal{D}_{rec}$ 
13:    $Acc \leftarrow \text{Test } f \text{ on } \mathcal{D}_{rec}$ 
14:   if  $Acc > Acc_{rec}$  then
15:     Break
16:   end if
17: end for
```

Our research shows that a very small $\mathcal{D}_{for}$, as randomly selected from the clean dataset for $f(\cdot)$, is adequate for almost completely removing backdoor effect from the model, that is, resulting in a negligible ASR (Section 5.1). The threshold Acc_{for} was set in our experiment to $\min(2/C, 0.6)$, where C is the number of classes in the primary task. The *recovery* dataset $\mathcal{D}_{rec}$ includes only correctly labeled data, which could be a subset of the testing data for the primary task. This ensures that the final model $\bar{f}$ after the *recovery* step achieves an accuracy close to that of the input model $f(\cdot)$, while keeping ASR exceedingly low (approaching 0). Note that there is no overlap between $\mathcal{D}_{rec}$ and $\mathcal{D}_{for}$. In our experiments, we set the threshold Acc_{rec} for the *recovery* step to 0.97 of the input model's accuracy.

4. Theoretical Analyses

To show why SEAM works and why it outperforms other state-of-the-art solutions, we present a theoretical analysis. We first model the backdoor attack, i.e., injection of a backdoor into a DNN so that the network will classify any

input sample with a trigger to a target class, as a multi-task learning (MTL) problem that aims at learning two distinct tasks simultaneously: the overt primary classification task and the covert trigger recognition (backdoor) task. In the SEAM pipeline, the *forgetting* step is to induce Catastrophic Forgetting (CF) on both primary and backdoor tasks by training the backdoored DNN model using randomly labeled datasets, and the *recovery* step is to restore the performance of the primary task (but not the backdoor task) by re-training the model after the CF on a small set of clean data for the primary task. Under the MTL model, we then analyze SEAM with the Neural Tangent Kernel (NTK) theory that approximates an NN as a kernel function together with a linear classification model in the kernel space [30]. This separation of the kernel function (for feature extraction) and classification is important for understanding the effectiveness of our random-labeling task, since it is meant to mostly impact the latter while largely preserving the former.

More specifically, our analysis shows that 1) the *forgetting* step incurs the most effective CF on the trigger recognition task when the trigger is unknown, and 2) the *recovery* step will not revive the “forgotten” backdoor task, as guaranteed by the competition between the primary classification task and the trigger recognition task. It is important to note that although a backdoored model is usually trained for the primary task (overt task) and the backdoor task (covert task) simultaneously, SEAM performs the forgetting step and the recovery step sequentially on a given model that could be a backdoored model (simultaneously trained for the backdoor and the primary tasks) or a benign model (trained only for the primary task). Therefore, our analysis is meant to evaluate the sequential operations of SEAM, regardless of whether the model SEAM works on is trained simultaneously on the backdoor and primary tasks.

4.1. NTK Modeling of Continual Learning

We use the NTK theory for continual learning in our theoretical analyses. We consider a sequence of tasks $\tau_1, \tau_2, \dots, \tau_T, T \in \mathbb{N}^*$, in which each is a supervised learning task with its input and output in the same high dimensional spaces, respectively. Consider the NN trained on the training data labeled for each of the T tasks in the sequential order. Prior research [12] utilizes NTK to approximate a NN trained for a target task τ_T with the model trained for a source task τ_S , where tasks τ_S and τ_T are any two tasks in a sequence, and τ_S occurs before τ_T . The NTK approximation from $f_{\tau_S}^*(x)$ to $f_{\tau_T}^*(x)$ is expressed as: $f_{\tau_T}^*(x) \approx f_{\tau_S}^*(x) + \langle \phi(x), \omega_{\tau_T}^* - \omega_{\tau_S}^* \rangle$, where $\omega_{\tau_S}^*$ is the final vector of weights after the training on task τ_S , $\langle \cdot \rangle$ represents the inner product in the kernel space. $\nabla_{\omega} f_0(x) = \phi(x) \in \mathbb{R}^q$ is defined as the NTK feature map for an input sample x , where q is the number of weights in the neural networks. Note that in prior research [12] and [4], the NTK is defined on the Taylor expansion around the initial state of the DNN (f_0), but it can be extended to Taylor expansion around any state of the DNN to approximate a subsequent state of DNN in sequential training.

4.2. Measuring Catastrophic Forgetting using NTK

Following, we first present the definition of CF, which is defined over the transition from a source task to a target one, as measured from a given dataset. Then we utilize NTK to describe the CF measurement, so we can separate an NN’s feature representation from classification. This is important for analyzing the impacts of SEAM on CF (e.g., how the forgetting task interferes with classification).

Definition of CF. The formal definition of the CF from the prior work [12].

Definition 1. Let τ_S and τ_T be the source and target tasks, respectively, where τ_S is trained before τ_T in a sequence of continual training tasks, and $D_{\tau_S} = (X_{\tau_S}, Y_{\tau_S})$ be the testing set of the source task. Then the CF of the model for the source task f_{τ_S} after the training of all the subsequent tasks until the target task τ_T w.r.t. the testing data D_{τ_S} is defined as:

$$\Delta^{\tau_S \rightarrow \tau_T}(X_{\tau_S}) = \sum_{x \in D_{\tau_S}} (f_{\tau_T}^*(x) - f_{\tau_S}^*(x))^2$$

where $f_{\tau_S}^*$ and $f_{\tau_T}^*$ represent the models (after training) for the source and target tasks, respectively. Throughout this paper, CF is always defined on three elements, the two tasks involved in the task transition: the source task and the target task, and a dataset X on which CF is measured. Note that the CF can be evaluated on any dataset X taken as the input to the source and target models; even for the same pairs of source and target models, the CF can be different on different input dataset X . Therefore, it is defined as a function, i.e., $\Delta^{\tau_S \rightarrow \tau_T}(\cdot)$. In the case of multi-class classification, we represent the predicted output as a one-hot vector in which $y_k = 1$ for the predicted class k and $y_i = 0$ for all other classes. To measure the CF on the transition of such tasks over a dataset X , we compute the squared norm of the one-hot vectors produced by the models for the source and the target tasks, respectively.

Impact on classification: residual. Interestingly, the above expression of CF is general and has a linear form under the NTK representation, which enables us to perform insightful analyses on the effectiveness of different forgetting approaches. For this purpose, we introduce Lemma 4.1, which is used by the prior study to measure CF [4]:

Lemma 1. Let $\{\omega_{\tau}^*, \forall \tau \in [T]\}$ be the weight at the end of the training of task τ . The CF of a source task τ_S with respect to a target task τ_T is measured on a data X as:

$$\Delta^{\tau_S \rightarrow \tau_T}(X) = \|\phi(X)(\omega_{\tau_T}^* - \omega_{\tau_S}^*)\|_2^2 \quad (1)$$

Further, as shown in the prior work [4], we have

$$\omega_{\tau_T}^* - \omega_{\tau_S}^* = \phi(X_{\tau_T})^\top \left[\phi(X_{\tau_T}) \phi(X_{\tau_T})^\top + \lambda I \right]^{-1} \tilde{y}_{\tau_T} \quad (2)$$

where $\tilde{y}_{\tau_T} = y_{\tau_T} - f_{\tau_S}^*(X_{\tau_T})$ is the residual between the true labels (i.e., the desirable outputs) and the predicted labels by the source model on the target task’s training data X^{τ_T} , which is a vector with the size of the number of samples in X^{τ_T} . λI is the regularization term for better lazy training to improve the precision of the Taylor expansion [7]. Note that the residual here describes the impact of the tasks on

the classification component of the DNN model while the remaining of Eq. 2 shows the changes to other components.

Impact on representation: task similarity. The combination of Eq. 1 and Eq. 2 leads to the following corollary, which is also provided by the prior study [12]. It measures the CF incurred by the transition from the task τ_S to the task τ_T w.r.t. D_{τ_S} :

Corollary 2. *The CF caused by the sequence of the tasks that end with τ_T to τ_S w.r.t. D_{τ_S} can be expressed as follows:*

$$\Delta^{\tau_S \rightarrow \tau_T}(X_{\tau_S}) = \left\| U_{\tau_S} \Sigma_{\tau_S} \underbrace{V_{\tau_S}^\top V_{\tau_T}}_{O_{\tau_S, T}} \Sigma_{\tau_T} [\Sigma_{\tau_T}^2 + \lambda I]^{-1} U_{\tau_T}^\top \tilde{y}_{\tau_T} \right\|_2^2 \quad (3)$$

Corollary 2 is extended from the Lemma 1 in paper [12]. This new form of the CF measurement lays the foundation for the analysis on both the forgetting and the recovery tasks. U, σ and V represent the left singular vector, singular value, and right singular vector respectively after SVD. Their subscripts represent the tasks they are corresponding to. In addition to the residual for the influence on classification, the term $\|O_{\tau_S, T}\|_2^2 = \|V_{\tau_S}^\top V_{\tau_T}\|_2^2$, which is positively related to the CF, is considered to be a good similarity metric between τ_S and τ_T in prior studies [12]: intuitively, a large $\|O_{\tau_S, T}\|_2^2$ indicates that the angle between the representation vectors produced by the models (for τ_S and τ_T) on the same input samples (i.e., X_{τ_S}) is small (i.e., these two tasks are similar). Given a fixed residual, a large similarity leads to a large CF, i.e., a high impact of τ_T on τ_S . In Eq. 3, $U_{\tau_S}, \Sigma_{\tau_S}$ and $V_{\tau_S}^\top$ result from the SVD of kernel matrix over the dataset X_{τ_S} ; the subscript of τ_S is used here to denote the dataset X_{τ_S} instead of the source task τ_S . Notably, once the model representation (i.e., the feature map) is fixed for a neural network, the term $O_{\tau_S, T}$ is dependent on the dataset X_{τ_S} (on which the CF is evaluated) as well as the training dataset X_{τ_T} for the target task. For the rest of the paper, we will refer to this term as the "task similarity" following the terminology in the previous studies [12] even though it is not directly related to the two tasks involved in the transition, but to their training datasets.

4.3. Analyzing Backdoor Forgetting with SEAM

Using NTK to model CF, we are able to prove that the random-labeling step of SEAM maximizes the residual, which is proportional to the CF, on a given training dataset. This result demonstrates that our forgetting task is the best we could do to disrupt the backdoor task in the absence of the information about the backdoor (trigger, source and target classes, etc.). Following, we elaborate the analysis.

Effectiveness of random labeling. Suppose that a source NN $f_{\tau_P}^*$ is trained on the task τ_P using a poisoned dataset $D_p = \{X_p, Y_p\}$. The model may perform both the primary classification task and the covert trigger recognition task, because D_p contains a subset of training samples carrying a trigger, denoted as $D_t = \{X_t, Y_t\}$ where every $y_t \in Y_t$ represents the same class (i.e., the target class). The goal of the backdoor forgetting is to train a *competitive* task τ_F

starting from the source model $f_{\tau_P}^*$ using a dataset D_{τ_F} such that the backdoor (i.e., the trigger recognition task) injected into the source is forgotten (i.e., unlearned) in the resulting model $f_{\tau_F}^*$. According to the NTK analyses described above (Corollary 2), for any input data X (X may be a normal input or a trigger-containing input), the CF from the source model for the task τ_P to the competitive model for the task τ_F measured on X is:

$$\Delta^{\tau_P \rightarrow \tau_F}(X) = \left\| \phi(X) \phi(X_{\tau_F})^\top [\phi(X_{\tau_F}) \phi(X_{\tau_F})^\top + \lambda I]^{-1} \tilde{y}_{\tau_F} \right\|_2^2 \quad (4)$$

where $D_{\tau_F} = (X_{\tau_F}, Y_{\tau_F})$ are the labeled training dataset for the task τ_F . Here, the kernel map $\phi(\cdot)$ is defined through the Taylor expansion around the weights of the source model ω_S^* , and remain the same throughout our theoretical analyses of the SEAM approach. Note that, on the right-hand side of Eq. 4, every term after $\phi(X)$ is independent of X , whereas only the residual term is dependent on the label (i.e., the desirable output) y_{τ_F} of the training data D_{τ_F} , and the other terms are dependent only on the input (i.e., y_{τ_F}) of the training data. As a result, we have the following lemma:

Lemma 3. *For any specific sample X , the CF from the source model $f_{\tau_P}^*$ to a competitive model trained on X_{τ_F} is proportional to the residual: $\tilde{y}_{\tau_F} = y_{\tau_F} - f_{\tau_P}^*(X_{\tau_F})$.*

Note that this residual is independent of the sample X on which the CF is evaluated. Therefore, we have the following theorem:

Theorem 4. *Given a fixed input of a training dataset X_{τ_F} , the randomly assigned wrong label y_{τ_F} maximizes the residual $\tilde{y}_{\tau_F}$, and thus maximizes the CF of any input sample X from the source model to the competitive model trained on the labeled dataset $D_{\tau_F} = (X_{\tau_F}, y_{\tau_F})$.*

Discussion. The proof of Theorem 4 is given in the Appendix. The theorem indicates that if we want to leverage a given dataset X_{τ_F} to train a *competitive* task τ_F that maximizes the CF of the task τ_P on any specific sample X , we should resort to the task that maximizes $\tilde{y}_{\tau_F}$, which can be achieved by using a random label y_{τ_F} that is different from their predicted label by the source model $f_{\tau_P}^*(X_{\tau_S})$. In practice, using a small subset of clean data $D_c = \{X_c, Y_c\}$, we may construct the training dataset $D_{\tau_F} = \{X_c, Y_F\}$ by generating a randomly wrong label for each sample $x \in X_c$ to replace Y_c . Notably, according to Theorem 4, given a fixed training dataset for the forgetting task D_{τ_F} (which contains only the clean data without triggers), the model trained on the dataset with random wrong labels will still induce the maximum CF on any sample X by the source model $f_{\tau_P}^*$, no matter whether X is a clean sample set or a trigger-containing sample set. As such, any backdoor task as well as the primary task will be forgotten by our approach. This is the best we can achieve when we do not know the trigger and the backdoor task. On the other hand, if the trigger is known (which is not realistic in a practical scenario), we may selectively unlearn the backdoor task.

Eq. 4 also implies that the direct fine-tuning of the model $f_{\tau_P}^*$ using the clean dataset D_C (equivalently, using

the primary task as the *competitive* task) will not cause effective unlearning of the backdoor, which is consistent with the findings reported by previous studies [39]. Specifically, during the fine-tuning, the true labels Y_c of the clean samples in $D_c = \{X_c, Y_c\}$ are used, and thus the residual $\tilde{y}_{\tau_F} = Y_{\tau_F} - f_{\tau_P}^*(X_{\tau_F})$ is very small, because the model $f_{\tau_P}^*$ is expected to correctly predict most of the true labels of the clean data for the primary task. Therefore, the CF induced by the *competitive* task of SEAM trained on randomly labeled training data. Intuitively, since the initial model $f_{\tau_P}^*$ already makes a correct prediction on most samples in the clean dataset, fine-tuning does not change the model significantly and thus the backdoor task may not be effectively unlearned.

Example. We conducted a simple experiment on CIFAR-10 to validate our theoretical analyses. Using a training dataset with 50,000 samples, including 2500 trigger-containing ones, we trained a VGG-16 NN with an ACC of 92% for the primary classification task and an ASR of 99% for a polygon trigger. We then ran the *forgetting* step of SEAM 10 different times on the NN by retraining the model on 1000 clean samples for *only one epoch* (batch size = 256), in which for each time, a different fraction (i.e., 100, 200, ..., 1000, respectively) of the 1000 clean inputs were assigned to randomly wrong labels; as a result, the residual term increased in these 10 experiments. The change of ACC and ASR in these 10 forgetting experiments are illustrated in Fig. 1. As expected from our theoretical analyses (Theorem 4), with the residual term continuing to increase, the ACC and the ASR both decrease to almost zero (i.e., the CF of both the primary and the backdoor tasks becomes maximum). Interestingly, the ASR decreases slightly faster than the ACC, suggesting that in practice, unlearning of the backdoor task might be done more effectively than unlearning of the primary task.

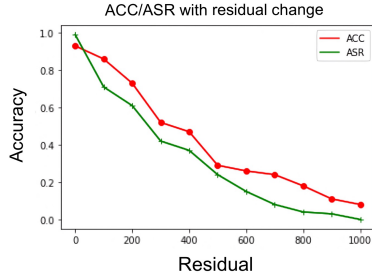


Figure 1: Relation of accuracy and residual: The decreasing ACC and ASR with the increasing residual term.

4.4. Analyzing Primary Task Recovery with SEAM

Our analysis using the NTK model of the forgetting task shows that SEAM can maximize the disruption (CF) of an unknown backdoor task for a given set of clean data for training the forgetting task. Following we further report our analysis on the recovery task using NTK.

Selective recovery. We denote the recovery task as τ_C , which is trained on a small subset of clean data $D_C = (X_C, Y_C)$ for the primary classification task. According to

Corollary 2, the CF of the *competitive* task τ_F on any dataset X after learning the recovery task τ_C is:

$$\Delta^{\tau_F \rightarrow \tau_C}(X) = \|U_X \Sigma_X V_X^\top V_{\tau_C} \Sigma_{\tau_C} [\Sigma_{\tau_C}^2 + \lambda I]^{-1} U_{\tau_C}^\top \tilde{y}_{\tau_C}\|_2^2 \quad (5)$$

where $U_X \Sigma_X V_X^\top$ represents the SVD of the kernel matrix of X . Note that the residual $\tilde{y}_{\tau_C}$ represents the difference between the given labels of the recovery data X_C and their labels predicted by the *competitive* task. Since the *competitive* task τ_F is trained on a randomly wrong label dataset D_{τ_F} , it always outputs a randomly wrong label for any given input sample. Also note that the norm of the residual, $\|\tilde{y}_{\tau_C}\|_2^2$, is expected to be independent (see details in Footnote 2, our extended proof online) of the target task τ_C , as long as the labels of X_{τ_C} are consistent with $f_{\tau_P}^*(X_{\tau_C})$, no matter whether it is the primary task or a backdoor task. Therefore, the following theorem can be proved following the previous work [12].

Theorem 5. *If τ_F is trained on a randomly labeled dataset, the CF of the task τ_F on any dataset X after training on the recovery data D_C is dependent on $\|O_{X,C}\|_2^2 = \|V_X^\top V_{\tau_C}\|_2^2$, i.e., the similarity between representation of the dataset X and the representation of the recovery dataset X_{τ_C} .*

According to the Theorem 5, if X is a subset of the testing dataset for the primary task (i.e., following the same distribution as D_C), the representations of the samples in X and D_C are highly similar, and thus $\|O_{X,C}\|_2^2$ is large, then the CF is large, which indicates the recovered model gives very different output from the initial model for the task τ_F (i.e., it effectively recovers the primary task). In contrast, if X is a dataset for the backdoor task, $\|O_{X,C}\|_2^2$ is small and thus the CF is small, which indicates the recovered model does not recover the backdoor task.

5. Evaluation

We evaluated the effectiveness of our SEAM pipeline on four datasets (Appendix section 2) against three representative backdoor attacks (Section 5.1), and also tested its efficiency in terms of its scalability and efficacy using only a very small set of clean data. We further compared our approach with three state-of-the-art unlearning approaches (Section 5.3) and demonstrated the robustness of SEAM against adaptive evasion (Section 5.4). This evaluation study is based upon the following metrics:

ACC for accuracy. ACC measures the ratio of clean inputs that can be correctly classified. In our study, ACCs are measured on the testing data of each dataset we used.

ASR for attack success rate. ASR measures the ratio of the inputs carrying triggers that can be misclassified into the target class, when they come from certain source class(es). In our experiments, ASRs are measured on all the images from the source classes in the testing set of each dataset.

FID for Fidelity. FID measures the gap between ACC and ASR achieved by a backdoor unlearning technique w.r.t. the original ACC of a backdoored model: $(ACC_s - ASR_s)/ACC_b$ where ACC_b is the model's ACC before the unlearning, and ACC_s and ASR_s are the ACC and ASR

after the unlearning. Essentially, FID is the normalized gap between ACC and ASR. The larger such a gap is, the more effective the unlearning process becomes in suppressing ASR and preserving ACC. We used four public datasets in our experiments: MNIST [37], CIFAR10 [36], GTSRB [55] and the dataset of NIST’s TrojAI Competition [28]. The details of these datasets we used are in Appendix section 2.

5.1. Efficacy

Here we present our experimental results on the efficacy of SEAM using the aforementioned datasets. Our approach is found to be highly effective in preserving the ACC of the original model and in the meantime suppressing the effects of backdoors when they are present in the model. In our experiments, we ensure that the testing set used to measure the performance of SEAM does not have any overlap with the clean dataset for inducing CF to the model and recovering its preliminary task, and also the clean set is selected independently from the training dataset (except our study on natural backdoor). In practice, however, the defender could utilize her testing dataset for evaluating the functionality of the model to blindly unlearn the backdoor from the model through SEAM.

On image tasks. We evaluated SEAM on the image tasks using MNIST, CIFAR10, GTSRB and the datasets of *Round 3-4* of the TrojAI competition. For each of the MNIST, CIFAR10 and GTSRB datasets, we trained two sets of four DNN models, including ShuffleNet, VGG16, ResNet18 and ResNet101, and ran each set under one of two representative backdoor attacks: Reflection [45], which is typical of backdoor injection through data poisoning, and TrojanNet [61], which is typical of model infection attacks [14]. On *Round 3-4* datasets of the TrojAI competition, backdoors with various polygon or filter triggers were injected into the models through data poisoning, in which a certain portion of trigger-carrying images were added to the training dataset of each victim model [28].

For the two representative backdoor attacks, Reflection involves selecting a set of candidate images and utilizing reflection transformation to generate triggers, and further injecting them into the training dataset of the victim model. In our experiments, following the steps provided by the Reflection paper [45], we sampled 200 images from the target class as candidate images and selected the reflection-transformed images with the highest ASRs on a dummy dataset (the training set in our experiments to give the adversary advantage) as triggers, and further pasted such triggers onto 10% of the selected images and injected them into the training dataset for each victim model. The TrojanNet attack trains a simple NN to recognize whether a trigger appears on the input images and then merges the NN and the victim model [61]. Again, following the experimental setting given by the prior research [61], we utilized a 4x4 square trigger with 11 white pixels and 5 black pixels, and a 4-layer perceptron NN as the trigger recognition network, and further integrated the network into the victim DNN by combining the outputs of their penultimate layers through interpolation. Note that both Reflection [45] and

TrojanNet [61] requires a source class label as their inputs for generating Trojaned models. We randomly selected a label as the source for each attack.

In our experiments on SEAM against Reflection and TrojanNet over MNIST, GTSRB and CIFAR10, we utilized 0.1% of the training data of each dataset for the *forgetting* step (label randomization) and 10% of its training data for the *recovery* step (retraining the randomized model on the primary task). Further, we leveraged the testing data provided by these datasets to measure ACC, and added triggers to all the testing inputs from the source class of each attack to measure ASR. Table 1 presents our experimental results. Here, A high ACC and a high ASR before SEAM characterize an effective backdoor attack. A high ACC and a low ASR after the unlearning, that is, a high FID, indicate effective suppression of the backdoor effect from a victim model. On all three datasets, SEAM is found to achieve a good FID against both attacks. The minimum FID is 95.24% for the Trojaned Vgg16 on CIFAR10 under TrojanNet, which is caused by the remaining ASR after unlearning (3.67%). Note that the ACC of Vgg16 on CIFAR10 is among the lowest before the unlearning operation, with and without the backdoor, which indicates the limitation of the model itself and could have an impact on the effectiveness of our approach on the model.

The datasets of TrojAI *Round 3-4* contain a large number of models (3198 models) in various model architectures and with different triggers (Section B). In our experiment, we utilized a very small set of clean data provided by the competition organizer for the *forgetting* and the *recovery* steps, which is only 0.1% of the data for training each model; we also followed the competition’s documents [28] to generate the testing data (a separate dataset about 10% the size of the training data, with those with the source label also used with the trigger for measuring ASR). Table 4 illustrates the experiment results. With this unprecedentedly small set of clean data, still SEAM achieved a FID of 89.85% and 86.75% for *Round 3* and *4*. Actually, only 3-5 clean samples from each class are provided for each model (Section 5.2). No existing technique, up to our knowledge, could utilize such a small set of clean data for meaningful unlearning (see Table 2).

On NLP tasks. We evaluated SEAM on NLP tasks using datasets of *Round 5-7* of the TrojAI competition. These sentiment classification models are built by stacking a classification model on top of pre-trained transformers [70] (e.g., BERT). During training, the transformers are fixed and only the weights of the classification models are updated, which mimics the popular pipeline of NLP tasks. In this way, the backdoor will only affect the classification model. The classification models in the datasets have three different architectures: GRU [8], LSTM [22] and Fully Connected (FC) networks, and use various hyper-parameters (e.g., different number of layers). Models of *Round 7* are trained for NER and built by stacking a linear layer upon 4 kinds of transformers (Section B). During the training, the weights of both the transformers and the linear layers are updated.

In our experiments, again, we utilized the small set

TABLE 1: Effectiveness of SEAM against backdoor attacks on MNIST, GTSRB and CIFAR10. Rf represents Reflection attack, Tj represents TrojanNet attack, ACC_b and ASR_b represents the ACC and ASR after attacks and before SEAM, ACC_s and ASR_s represents the ACC and ASR after SEAM, FID represents the *Fidelity* $\frac{ACC_s - ASR_s}{ACC_b}$. SEAM forgets on 0.1% training data and recovers on 10% training data.

Dataset	Model	ACC	ACC_b (before SEAM)		ASR_b (before SEAM)		ACC_s (after SEAM)		ASR_s (after SEAM)		FID	
			Rf	Tj	Rf	Tj	Rf	Tj	Rf	Tj	Rf	Tj
MNIST	ShuffleNetx1.0	99.14%	99.16%	99.72%	100%	100%	98.05%	97.28%	0.91%	0%	97.96%	97.55%
	Vgg16	99.38%	99.37%	99.00%	100%	100%	97.07%	97.03%	0.78%	0%	96.90%	98.01%
	ResNet18	99.69%	99.35%	98.57%	100%	100%	98.30%	98.21%	0.76%	0%	98.18%	99.63%
	ResNet101	99.63%	98.39%	98.20%	100%	100%	97.88%	97.52%	0.83%	0%	98.64%	99.31%
GTSRB	ShuffleNetx1.0	99.72%	97.07%	99.78%	99.68%	100%	95.03%	97.57%	0.66%	0.71%	97.22%	97.07%
	Vgg16	97.67%	94.70%	98.37%	99.98%	100%	92.97%	96.34%	0.81%	0.66%	97.32%	97.27%
	ResNet18	99.85%	94.29%	98.56%	99.98%	100%	93.86%	97.21%	0.75%	0.63%	98.75%	97.99%
	ResNet101	99.83%	97.94%	98.33%	100%	100%	95.64%	96.98%	0.92%	0.89%	96.71%	97.72%
CIFAR10	ShuffleNetx1.0	94.63%	90.60%	94.41%	100%	100%	90.02%	92.64%	1.57%	2.34%	97.63%	95.65%
	Vgg16	95.12%	91.62%	95.11%	99.95%	100%	90.99%	94.25%	1.17%	3.67%	98.04%	95.24%
	ResNet18	96.50%	93.09%	96.50%	100%	100%	92.07%	96.01%	2.10%	3.17%	96.65%	96.21%
	ResNet101	96.98%	91.24%	96.98%	100%	100%	90.79%	95.82%	2.22%	2.55%	97.07%	96.17%

TABLE 2: Comparison of SEAM with backdoor defenses on MNIST, GTSRB and CIFAR10. NC represents Neural Cleanse defense, FP represents Fine Pruning defense. SEAM forgets on 0.1% training data and recovers on 10% training data.

Dataset	Model	Reflection						TrojanNet					
		<i>Fidelity (FID)</i>			Time (seconds)			<i>Fidelity (FID)</i>			Time (seconds)		
		NC	FP	SEAM	NC	FP	SEAM	NC	FP	SEAM	NC	FP	SEAM
MNIST	ShuffleNetx1.0	14.64%	89.50%	97.96%	825	575	104	100%	89.13%	97.55%	818	632	112
	Vgg16	< 0%	87.95%	96.90%	793	721	229	90.03%	89.22%	98.01%	802	701	225
	ResNet18	72.75%	90.17%	98.18%	1163	909	329	100%	100.97%	99.63%	1148	938	341
	ResNet101	< 0%	90.89%	98.64%	7865	4445	935	89.61%	90.27%	99.31%	7361	4415	1018
GTSRB	ShuffleNetx1.0	96.72%	99.65%	97.22%	3442	1040	218	99.97%	86.35%	97.07%	3648	770	217
	Vgg16	96.36%	100.25%	97.32%	2634	909	413	99.41%	96.05%	97.27%	2735	697	423
	ResNet18	98.49%	99.98%	98.75%	3424	792	601	99.43%	93.95%	97.99%	3523	874	639
	ResNet101	99.04%	99.48%	96.71%	13287	4990	2052	99.47%	97.28%	97.72%	13948	5000	2412
CIFAR10	ShuffleNetx1.0	90.79%	82.03%	97.63%	1677	1033	231	89.23%	86.95%	96.15%	1725	1131	253
	Vgg16	57.17%	85.24%	98.04%	1232	966	425	89.23%	88.36%	95.24%	1277	987	443
	ResNet18	88.96%	85.26%	96.65%	1662	1293	686	89.44%	97.19%	96.15%	1695	1404	679
	ResNet101	89.52%	92.25%	97.07%	7763	5114	2910	89.67%	95.19%	96.17%	7277	5283	3008

of clean data (0.1% of the training set) provided by the competition organizer for the *forgetting* and the *recovery* steps; we also followed the competition’s documents [28] to generate the testing data. Table 4 illustrates the experimental results. As we can see from the table, SEAM achieves an average FID of 88.30%, 89.16% and 92.65% on the datasets of *Round 5* and *Round 6* and *Round 7* respectively. The relatively higher *Fidelity* in the last round could be attributed to the higher ACC of the infected models in the round (93.60% vs. an average 90.15% of the models in other rounds), and the availability of more clean training data: in the sentiment analysis (*Round 5-6*), one clean sentence released by the competition organizer can only be used as one instance in the training set (for both the *forgetting* and *recovery* operations), while the same sentence can be broken down to multiple entity-related terms for unlearning backdoors in an NER model, the *Round 7* task.

On clean model. To understand the impacts of SEAM on the accuracy of the clean model, we performed experiments on CIFAR10, with the results reported in Table 3, i.e., the change of the clean models’ ACC before and after running SEAM. In the experiments, we again utilized a small set of clean data (0.1% of the training set) for the *forgetting* and the *recovery* steps. As we can see from Table 3, an average accuracy loss caused by SEAM is just 1% on the clean models with four mainstream structures.

5.2. Efficiency

We further analyzed the efficiency of SEAM from two aspects: execution time and clean data size.

TABLE 3: The task accuracies of the clean models before and after processed by SEAM

	Before	After
ShuffleNetx1.0	94.63%	93.14%
Vgg16	95.12%	94.42%
ResNet18	93.09%	92.18%
ResNet101	96.98%	95.73%

TABLE 4: Comparison of SEAM with backdoor defenses on Trojai competition *Round 3-7*. NC represents Neural Cleanse defense, FP represents Fine Pruning defense. Results are averaged among all backdoor infected models in each round. SEAM forgets and recovers on the clean dataset provided for each model (about 0.1% of the training dataset). The results are averaged among all models for each round. The unit of time is second.

		Round 3	Round 4	Round 5	Round 6	Round 7
FID	NC	53.42%	54.15%	67.27%	64.18%	61.90%
	FP	71.33%	72.81%	64.49%	63.39%	67.13%
	SEAM	89.84%	88.75%	88.30%	89.16%	92.65%
Time	NC	5218	4733	2763	2893	4919
	FP	2107	2053	2378	2439	2643
	SEAM	39	39	41	47	49

Execution time. Overall, SEAM is found to be highly efficient, vastly outperforming other unlearning techniques (see Table 2 and 4) in execution time. Particularly, from Table 2, we can see that on various models trained on the three popular datasets (MNIST/GTSRB/CIFAR10), SEAM typically just needs no more than 12 minutes to nearly completely remove the backdoor effect. The only exception is ResNet101, which uses massive GPU memory, so we had to reduce its batch size to 8, instead of 32 set for other model architectures.

Then we take a close look at the *forgetting* and *recovery* steps. The time complexities of these steps are $\mathcal{O}(N_{for})$ and $\mathcal{O}(N_{rec})$, where the former is the number of epochs for training the random-labeling task of the *forgetting* step, and the latter is the number of epochs for training the primary task for the *recovery* step. We compared the execution time of SEAM on infected ResNet18 models on three datasets (MNIST, GTSRB and CIFAR10) with these models' original training time. As we can see from the results in Table 6, on average, SEAM takes less than 7% of the time for training a model from scratch to suppress the model's backdoor effect while preserving its legitimate classification capability.

We found that, on MNIST/GTSRB/CIFAR10, the *forgetting* step takes much less time than the *recovery* step, since 1) the *forgetting* typically needs less than 10 epochs while the *recovery* requires at most 100 epochs, and 2) the dataset for the *forgetting* is just 0.1% of the training dataset, while the dataset for the *recovery* was set to 10% in our experiments on MNIST/GTSRB/CIFAR10. On Round 3-7 of TrojAI competition, the *recovery* dataset is only 0.1% training dataset that largely accelerated the *recovery* step with the cost of lightly reduced *Fidelity*. The effect of clean data size will be evaluated later.

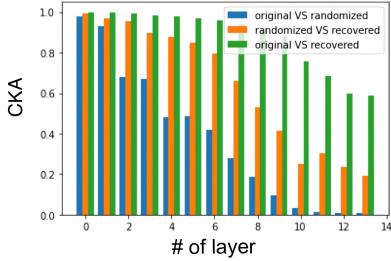


Figure 2: CKA on each layer of VGG16. Layer 0 is the first layer (input) and layer 14 is the last second layer.

To understand what makes SEAM so efficient, we looked into the changes of each layer after the *forgetting* step and the *recovery* step. Specifically, for a VGG16 model, we measured the similarity of the same layer between the original model (the backdoored one before unlearning), the randomized model (after the *forgetting* step) and the recovered model (after the *recovery* step). Here we use the centered kernel alignment (CKA) [35] as the metric, which ranges in $[0, 1]$ with $CKA = 1$ being identical and $CKA = 0$ being totally different. Fig. 2 demonstrates the CKA results for a VGG16 model on a clean dataset. As we can see here, for each layer, the closer it is located to the output, the more different can be seen between the original model and the randomized model. This indicates that many features of the original model has been preserved during the *forgetting* step, particularly those on the shallow layers of the model, so the *recovery* step only needs to restore the features on the layers more toward the output layer, thereby allowing a faster unlearning and recovering.

Clean data size. As mentioned earlier, we used 10% of the training data for recovering on MNIST/GTSRB/CIFAR10, and only 0.1% of the training data on Round 3-7 datasets of

the TrojAI competition. As a result, SEAM runs much faster on the Round 3-7 datasets, less than one minute on average for each model, though the *Fidelity* of the unlearning goes down a little bit ($> 88\%$ vs. $> 95\%$).

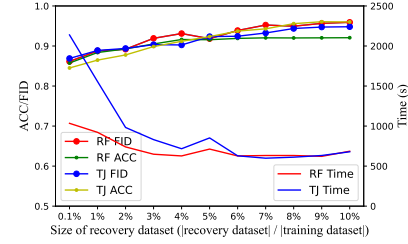


Figure 3: Running time, *Fidelity* (FID) and accuracy (ACC) of the recovered models. Here RF represents the effects of SEAM against the Reflection attack, and TJ represents the effects of SEAM against the TrojanNet attack.

To understand the impact of the size of *recovery* datasets D_{rec} on the effectiveness of unlearning, we ran SEAM on D_{rec} of different sizes (randomly drawn from CIFAR10). The results are presented in Fig. 3. We observe that the execution time of SEAM goes down along with the increase of the size of D_{rec} against Reflection attacks and TrojanNet attack. while the *Fidelity* changes gently. Again, we confirm that indeed SEAM only needs an exceedingly small amount of clean data (0.1%, about 5 images per class) to achieve a decent unlearning effect (*Fidelity* of 86%).

5.3. Comparison

We compared SEAM with Neural Cleanse (NC) [66], Fine-Pruning (FP) [39], fine-tuning, naive continuous-training, and Neural Attention Distillation (NAD) [38], five representative unlearning techniques. As mentioned earlier, NC is a detection-based approach that first recovers the trigger from a backdoored model and then removes the backdoor from the model through unlearning (i.e., retraining the model on the trigger-carrying inputs with the correct labels). FP is a blind unlearning approach that prunes a backdoored model and then fine-tunes it on a clean dataset, in an attempt to remove the backdoor effect. Fine-tuning is a method that tunes the parameters of a backdoored model's last two layers on a small set of clean data using gradient descent; continuous-training keeps on training the model on a small set of clean data, hoping to weaken its backdoor effect when it is infected. NAD is a blind unlearning approach that uses a clean dataset to distillate a new clean model from the victim model.

For a fair comparison, we implemented SEAM under the TrojanZoo framework [49] and utilized the implementations of NC and FP provided by the TrojanZoo team. We also kept the unlearning datasets for NC, FP and SEAM to the same size, i.e., 10% of the training datasets for MNIST/GTSRB/CIFAR10 and 0.1% for the TrojAI competition. Also, for Fine-pruning, we followed the FP paper [39] to set the prune ratio to 0.82 of all neurons (i.e., maximum 82% neurons would be pruned) and the fine-tuning epochs to 300 for MNIST, GTSRB and CIFAR10, 1000 for TrojAI competition Round 3-7.

Table 2 and Table 4 show the comparison of *Fidelity* among these solutions against the Reflection and TrojanNet attacks on MNIST/GTSRB/CIFAR10 and TrojAI datasets. We observe that SEAM achieves a high and stable *Fidelity* against both attacks on all eight datasets, while FP performs well against both attacks on MNIST and GTSRB, and NC only does well on GTSRB. The failure of NC against the Reflection attacks on MNIST and CIFAR10 could be attributed to its limitation in finding large triggers, as the Reflection attack may use an entire clean image as the trigger. The failure of FP against the Reflection attacks on CIFAR10 could be due to the difficulty in reducing ASR when trigger-relevant neurons have not been completely pruned.

A larger *Fidelity* gap between SEAM and NC/FP can be observed on *Round 3-7* datasets of the TrojAI competition. The small clean dataset (0.1% of training data) available for unlearning significantly reduces the efficacy of NC/FP. Specifically, on *Round 3-7*, the maximum *Fidelity* achieved by NC is 67.27% on the *Round 5* data, and for FP, it is 72.81% on the *Round 4* data, which are far below the performance of SEAM on the same datasets: 88.30% on *Round 5* and 88.75% on *Round 4*. Table 2 and Table 4 also compare the execution times of these solutions. On MNIST/GTSRB/CIFAR10, SEAM takes on average 1/6 of the execution time used by NC and 1/4 by FP. On *Round 3-7* of the TrojAI competition, SEAM takes on average just 1% of the execution time for NC and 1/50 for FP. The high efficiency of SEAM can be attributed to the fact that the *recovery* step only needs to restore the features on the layers close to the output (Section 5.1).

SEAM also significantly outperforms NAD and two simple baselines (fine-tuning and continuous training), especially when the clean data is scarce. In Table 5, we present the *Fidelity* results on the ResNet18 models for comparing SEAM, NAD and the two baselines on different sizes of clean datasets against the TrojanNet attack.

TABLE 5: The *Fidelity* results of NAD, two base-line unlearning methods (Fine-tune and Continuous-training) and SEAM using the clean datasets of different sizes (compared with the training data size).

	MNIST	GTSRB	CIFAR10
SEAM (10% of training data size)	96.21 %	97.99 %	96.21 %
NAD (10% of training data size)	95.34 %	90.15 %	81.24 %
Fine-tune (10% of training data size)	51.12 %	47.32 %	44.29 %
Continuous-training (10% of training data size)	50.32 %	49.97 %	40.11 %
SEAM (1% of training data size)	96.31 %	94.72 %	89.04 %
NAD (1% of training data size)	64.57 %	59.35 %	56.35 %
SEAM (0.1% of training data size)	91.15 %	88.89 %	85.04 %
NAD (0.1% of training data size)	30.04 %	21.08 %	18.61 %

As we can see from the table, NAD could not reasonably reduce ASR while maintaining a decent ACC when the size of the clean dataset goes down to 1% and further to the 0.1% of the training dataset size (note again that the clean data is not a subset of the training data). On the other hand, SEAM maintains its high effectiveness in unlearning, achieving 85% to over 91% *Fidelity* with a small set of clean data (0.1% of the training data size), in line with its performance on the TrojAI datasets, where for each model, only 10 samples are available for each class (Table 4).

Compared with another naive baseline – retraining the whole model from scratch on clean data, SEAM also demonstrates superior performance. We compared the performance of ResNet18 models recovered by SEAM and those trained from scratch on three datasets (MNIST, GTSRB and CIFAR10). As shown in Table 6, the models recovered by SEAM achieve on average 17% higher ACC than the models trained from scratch on the same clean dataset (10% of the whole training data size), and approach the ACC of the models trained from scratch on the whole training dataset. Also unlearning through SEAM takes about 45% of the time for the training from scratch to converge on 10% of the training data size (with much lower ACC) and only less than 7% of the time for training on the whole dataset.

TABLE 6: Comparison of *ACC* and time cost between the ResNet18 models recovered by SEAM and those trained from scratch.

		MNIST	GTSRB	CIFAR10
Train from scratch (same size as training data)	ACC	94.43%	97.31%	92.21%
	Time	~3h	~1.5h	~3h
Train from scratch (10% of training data size)	ACC	31.25%	72.12%	71.79%
	Time	~0.4h	~0.3h	~0.4h
SEAM (10% for recovery, 0.1% for forgetting)	ACC	92.79%	97.16%	91.43%
	Time	~0.1h	~0.2h	~0.2h

5.4. Evasion

In this section, we investigate several possible evasion methods against SEAM, including the Label Consistent (LC) backdoor [65], the Latent Separability (LS) backdoor, the Natural Backdoor (NB), the Entangled Watermarks (EW) [32] and the evasion polluting the recover dataset with trigger-carrying inputs.

TABLE 7: Effectiveness of SEAM against possible evasion methods on CIFAR10. ACC_b and ASR_b represents the ACC and ASR after attacks and before SEAM, ACC_s and ASR_s represents the ACC and ASR after SEAM, FID represents the *Fidelity* $\frac{ACC_s - ASR_s}{ACC_b}$. SEAM forgets on 0.1% training data and recovers on 1% training data. NB-t row represents results of SEAM forgets on 0.1% training data and recovers on 10% testing data. As the NB-t has same ACC_b , ASR_b , and very similar (less than 1% difference) ACC_s from NB, we only report ASR_s for NB-t.

		ShuffleNetx1.0	Vgg16	ResNet18	ResNet101
ACC_b	LC	90.54%	91.28%	93.82%	92.74%
	LS	90.14%	90.86%	91.61%	92.11 %
	NB	94.63%	95.12%	96.50%	96.98%
ASR_b	LC	98.06%	99.87%	99.78%	87.70%
	LS	98.17%	98.68%	99.12 %	97.83 %
	NB	84.73%	76.59%	83.24%	71.93%
ACC_s	LC	91.21%	91.03%	94.18%	92.53%
	LS	90.02%	90.13%	90.93%	91.05 %
	NB	94.27%	95.18%	95.64%	91.99%
ASR_s	LC	7.19%	7.93%	9.21%	9.34%
	LS	23.02%	22.51%	21.56%	22.41%
	NB	54.23%	43.15%	57.80%	59.61%
NB-t	LC	33.79%	28.73%	31.28%	34.40%
	LS	92.80%	91.04%	90.57%	89.70%
	NB	73.33%	74.42%	75.72 %	74.52%
FID	LC	42.31%	54.70%	39.21%	33.39%
	LS				
	NB				

Label Consistent backdoor. Label Consistent (LC) is a data poisoning backdoor attack aiming to inject a targeted backdoor that makes the victim model misclassify samples in a specific source class to a target class. The idea is to use correctly labeled yet trigger-carrying images (which

can escape human inspection) to cause the prediction of the target label to heavily rely on the triggers [65], so the triggers can be used to induce misclassification.

To find out the robustness of SEAM against LC, we performed a set of experiments on CIFAR10. Table 7 shows that our approach successfully reduces the ASR from $> 90\%$ to $< 10\%$ and recovers the ACC to the level similar to that of the original model. In the meantime, indeed LC weakens the effectiveness of SEAM by causing a higher ASR after the *recovery* step, compared with the results of running SEAM against the Reflection and TrojanNet attacks on CIFAR10 (Table 1). Note that unlike other data poisoning attacks, LC requires information about the target model’s representation space, which can only be estimated through model transferability. It is still less clear how likely transferring a backdoor in this way could succeed.

Latent Separability backdoor. The Latent separability (LS) backdoor is an emerging attack that aims to build a backdoored model by producing indistinguishable representations in the latent space for trigger-carrying inputs and clean inputs. As a result, the backdoor task will behave similarly as the primary task, making it harder to unlearn the backdoor without affecting the primary task. In particular, we evaluated SEAM against the Adaptive-Blend [50] attack on CIFAR10 with four typical model architectures (ShuffleNetx1.0, Vgg16, ResNet18, and ResNet101). The *LS* rows in Table 7 show the experimental results. We observe that, after being processed by SEAM, the backdoored models retain a high accuracy ($ACC_s \sim 90\%$) for their primary task but are significantly weakened in terms of their backdoor effects (with the Attack Success Rate $ASR_s \sim 22\%$). These results demonstrate the effectiveness of SEAM in defending against the attacks with the capability to undermine unlearning.

Natural backdoor. NB is the backdoor naturally generated during training, without the interference of a malicious party. It is introduced by the imperfection of the model architecture, the training process, or the training data, etc. Injection of NB is a process less stable than the poisoning attack: two independent training of the same model on the same dataset may lead to different NBs (with different triggers).

To evaluate the performance of SEAM against NB, we performed experiments on CIFAR10. Specifically, we first recovered the NB of the target model through trigger inversion [66]. Then, we ran SEAM on the target model using a forgetting dataset and a recovery dataset, with 0.1% and 1% of the training data considered to be “clean” (without the recovered trigger), respectively. The results are shown in Table 7. We observe that SEAM reduces the ASR of the NB, with the maximum reduction of 33.44% on VGG16. To further weaken the effect of NB, we utilized 10% of the testing dataset (which amounts to 1% of the training data in size but has no overlap with its content), for recovery. Then, we leveraged the remaining 90% of the testing data to measure the ACC and the ASR of the unlearned model. The results are presented by the NB-t row in Table 7, which indicates that the use of the clean data not in the training set for recovery could be more effective in suppressing NB.

Entangled watermarks. EW injects a backdoor as a watermark for ownership protection into the target model. The idea is to make the backdoor entangled with the primary task, so removal of the backdoor will undermine the target model’s capability to perform its primary task. To understand whether SEAM still work on the models infected by EW, we conducted experiments over these models trained on the MNIST and CIFAR100 datasets. Particularly, we utilized the source code of EW to generate the infected models and ran SEAM on them, using a clean dataset with the size of 0.1% of the training data (for these models) for forgetting and a clean dataset with the size of 1%-10% of the training data for recovery. Fig. 4 and Fig. 5 show the results. As we can see from the figures, when the size of the recovery dataset is exceedingly small, noticeable performance degradation on benign inputs can still be observed from the infected models processed by SEAM, with the impact more conspicuous on the small task such as MNIST. Specifically, the recovered ACC becomes $\leq 95\%$ when the size of the recovery dataset $\leq 6\%$ of the training set on MNIST, and the ACC becomes $\leq 54\%$ when the size of the recovery dataset $\leq 5\%$ of the training set on CIFAR100. However, with moderate increase in the recovery data size, SEAM is found to be able to quickly restore the ACC: when the size of recovery dataset become $\geq 10\%$ ($\geq 8\%$) of the training set, the ACC of the recovered model gets back to or even goes beyond the ACC of the original model, that is, 99% (60%) on MNIST (CIFAR100). To further investigate how SEAM unlearns the EW-injected backdoor, we analyze the change of each layer within the target model under SEAM through a CKA experiment on CIFAR100 (see Fig. 8 in Appendix 3).

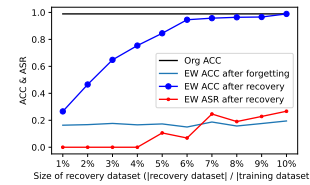


Figure 4: SEAM against EW on MNIST.

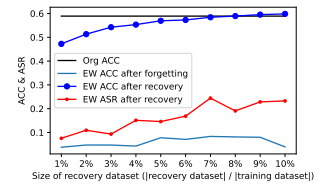


Figure 5: SEAM against EW on CIFAR100.

Polluted recovery dataset. To study the effect of data poisoning on SEAM’s recovery step, we performed a k-out-of-n experiment, where we first polluted the recovery dataset so a portion of it are trigger-carrying inputs, according to a poisoning ratio, and then evaluated the impacts of the

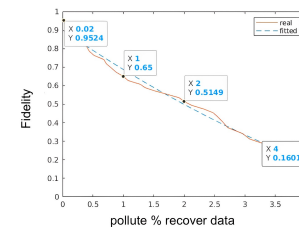


Figure 6: The *Fidelity* slowly decreases with the increasing trigger-carrying samples in the recovery dataset.

pollution on the protection of SEAM in terms of *Fidelity*. From Fig. 6, we observe that *Fidelity* exhibits a negative correlation with the poisoning ratio of the recovery dataset. Specifically, when the poisoning ratio is 0.02% (the recovery dataset including only 1 trigger-carrying input), the average *Fidelity* achieved by SEAM on the models decreases from 98.2% to 95.2%. When the poisoning ratio becomes 1% (i.e., the recovery dataset containing 50 trigger-carrying inputs), the *Fidelity* further decreases to 65%. These results demonstrate that SEAM has some resilience to data poisoning in the recovery dataset.

6. Related Work

There are mainly three categories of backdoor detection techniques: trigger reversion, model diagnosis and forensic analysis. In trigger reversion, the defender reconstructs the trigger using her own knowledge about the backdoor as constraints to guide trigger searching. For instance, Neural Cleanse [66] limits its trigger searching to find those with small norms, and I-BAU [77] approximates the implicit hyper-gradient for trigger reversion. Many proposed backdoor defense techniques [2], [13], [20], [53], [72] are in this category, based upon different searching algorithms. They are time-consuming, though, when compared with SEAM. Model diagnosis looks into the difference between backdoor infected models and benign models. These approaches focus on designing a metric to measure whether a model is closer to a backdoor infected model or a benign model. For example, ABS [42] utilizes a neuron activation vector and MNTD [75] takes the topological prior and the outputs of a meta-learned model for detecting backdoored models. These approaches are efficient but given the complexity of the neural network, they could be less accurate. Forensic analysis is meant to analyze the training data or the operation trace of a model to capture its backdoor behaviors or the attempt to inject a backdoor to the model [15], [60], [64]. An example is SCAn [60], which leverages the observation that the backdoor-carrying inputs and benign inputs from the source class have significant differences in the representation space. A statistical test can then differentiate them with a theoretical guarantee. A weakness of the approach is requirement for the presence of trigger-carrying inputs.

Compared with the detection approaches, another line of research is to remove the backdoor from the model, either using recovered backdoor triggers [19] or through blind unlearning [38]. SEAM is a blind unlearning technique, which we demonstrate to be more effective and efficient than existing approaches, as elaborated in § 5.3.

7. Discussion

Limitations. Our SEAM is meant to disable a backdoor within an infected model through partially unlearning it, rather than completely remove the backdoor from the model. Particularly, the partial unlearning performed by SEAM suppresses backdoor effects of an infected model in a blind and efficient way, instead of removing all backdoor traces from the model. As shown on Fig. 2, our approach breaks the trigger activation chain injected by the adversary into the deep layers (close to the output) of an ML model while

largely retaining the model’s features in its shallow layers (close to the input). As a result, it may be possible for the adversary to revive the dormant backdoor within the unlearned model, particularly when the model is fine-tuned on the dataset with trigger-carrying inputs. To understand the risk, we constructed experiments to investigate how many trigger-carrying inputs need to be injected into the fine-tuning dataset can revive the dormant trigger by our SEAM. We defer the experimental details in Appendix 3. The results (Fig. 7) demonstrate that the adversary can revive an effective backdoor only when he can pollute $\geq 4\%$ fine-tuning data, which however is less possible according to our threat model.

Fundamentally, SEAM leverages the architectural property of today’s mainstream DNNs, which utilize the same architecture to learn multiple tasks. So, if the adversary manages to separate the backdoor task from the primary task on the architectural level, our protection could fail. For example, one could train a model with two separate DNNs, one for the primary task and the other for the backdoor task [18]. The model switches between these two networks based upon the trigger pattern recognized from the input. Although this attack can indeed defeat our unlearning, it requires the full control on the training process and therefore cannot be executed through data poisoning. Also a direct combination of two models makes the model architecture differ significantly from the standard ones, rendering the attack easy to detect [18]. Further research is needed to understand whether other more effective poisoning attacks exist to pose a credible threat to our approach.

Future work. SEAM is meant to be a blind unlearning technique. However, its performance could be improved by leveraging the prior knowledge of a backdoor: e.g., the information about the trigger pattern could help design a more precise forgetting step, which strategically randomizes the labels of a subset of inputs, so as to speed up the step and enhance the ACC the recovery step could achieve. Further, an improvement of SEAM could enable the *forgetting* step to keep track of the speed of degradation for each class, which can be leveraged by the *recovery* step to retrain the model more heavily on selected classes, to make the unlearning process more efficient. Essentially, the *forgetting* step could be viewed as an attempt to find a good initialization point for learning the primary task. This implies that a good initialization may help reduce the ASR of a certain backdoor, which is an open problem for further research.

8. Conclusion

We present SEAM, a novel and high-performance blind unlearning technique for disabling backdoor, and analyzed its effectiveness through experimental studies and theoretic analysis. Our analysis shows that our *forgetting* step actually maximizes the CF on an unknown backdoor in the absence of triggered inputs. Through extensive experiments, we demonstrated efficacy and efficiency of SEAM on eight datasets with various model architectures against two representative attacks. The results show that SEAM outperforms existing defenses and achieves a high *Fidelity* efficiently.

9. Acknowledgment

We would like to thank the anonymous reviewers for their insightful comments. This work is partially supported by of IARPA's TrojAI project (Grant No. W91NF-20-C-0034).

References

- [1] Papers with code - the latest in machine learning. <https://paperswithcode.com/>.
- [2] William Aiken, Hyoungshick Kim, Simon Woo, and Jungwoo Ryoo. Neural network laundering: Removing black-box backdoor watermarks from deep neural networks. *Computers & Security*, 106:102277, 2021.
- [3] Sanjeev Arora, Simon S Du, Wei Hu, Zhiyuan Li, Russ R Salakhutdinov, and Ruosong Wang. On exact computation with an infinitely wide neural net. *Advances in Neural Information Processing Systems*, 32, 2019.
- [4] Mehdi Abbana Bennani, Thang Doan, and Masashi Sugiyama. Generalisation guarantees for continual learning with orthogonal gradient descent. *arXiv preprint arXiv:2006.11942*, 2020.
- [5] Bryant Chen, Wilka Carvalho, Nathalie Baracaldo, Heiko Ludwig, Benjamin Edwards, Taesung Lee, Ian Molloy, and Biplav Srivastava. Detecting backdoor attacks on deep neural networks by activation clustering. *arXiv preprint arXiv:1811.03728*, 2018.
- [6] Siyuan Cheng, Yingqi Liu, Shiqing Ma, and Xiangyu Zhang. Deep feature space trojan attack of neural networks by controlled detoxification. *arXiv preprint arXiv:2012.11212*, 2020.
- [7] Lenaic Chizat, Edouard Oyallon, and Francis Bach. On lazy training in differentiable programming. *arXiv preprint arXiv:1812.07956*, 2018.
- [8] Kyunghyun Cho, Bart Van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. On the properties of neural machine translation: Encoder-decoder approaches. *arXiv preprint arXiv:1409.1259*, 2014.
- [9] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009.
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Tamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pages 4171–4186. Association for Computational Linguistics, 2019.
- [11] Khoa Doan, Yingjie Lao, Weijie Zhao, and Ping Li. Lira: Learnable, imperceptible and robust backdoor attacks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11966–11976, 2021.
- [12] Thang Doan, Mehdi Abbana Bennani, Bogdan Mazouze, Guillaume Rabusseau, and Pierre Alquier. A theoretical analysis of catastrophic forgetting through the ntk overlap matrix. In *International Conference on Artificial Intelligence and Statistics*, pages 1072–1080. PMLR, 2021.
- [13] Yinpeng Dong, Xiao Yang, Zhijie Deng, Tianyu Pang, Zihao Xiao, Hang Su, and Jun Zhu. Black-box detection of backdoor attacks with limited information and data. *arXiv preprint arXiv:2103.13127*, 2021.
- [14] Yansong Gao, Bao Gia Doan, Zhi Zhang, Siqi Ma, Jiliang Zhang, Anmin Fu, Surya Nepal, and Hyoungshick Kim. Backdoor attacks and countermeasures on deep learning: A comprehensive review. *CoRR*, abs/2007.10760, 2020.
- [15] Yansong Gao, Change Xu, Derui Wang, Shiping Chen, Damith Chinthana Ranasinghe, and Surya Nepal. STRIP: a defence against trojan attacks on deep neural networks. In David Balenson, editor, *Proceedings of the 35th Annual Computer Security Applications Conference, ACSAC 2019, San Juan, PR, USA, December 09-13, 2019*, pages 113–125. ACM, 2019.
- [16] Siddhant Garg, Adarsh Kumar, Vibhor Goel, and Yingyu Liang. Can adversarial weight perturbations inject neural backdoors. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pages 2029–2032, 2020.
- [17] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. Badnets: Evaluating backdooring attacks on deep neural networks. *IEEE Access*, 7:47230–47244, 2019.
- [18] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. Badnets: Evaluating backdooring attacks on deep neural networks. *IEEE Access*, 7:47230–47244, 2019.
- [19] Wenbo Guo, Lun Wang, Xinyu Xing, Min Du, and Dawn Song. Tabor: A highly accurate approach to inspecting and restoring trojan backdoors in ai systems. *arXiv preprint arXiv:1908.01763*, 2019.
- [20] Wenbo Guo, Lun Wang, Yan Xu, Xinyu Xing, Min Du, and Dawn Song. Towards inspecting and eliminating trojan backdoors in deep neural networks. In *2020 IEEE International Conference on Data Mining (ICDM)*, pages 162–171. IEEE, 2020.
- [21] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*, pages 770–778. IEEE Computer Society, 2016.
- [22] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [23] Eduard Hovy, Mitchell Marcus, Martha Palmer, Lance Ramshaw, and Ralph Weischedel. OntoNotes: The 90% solution. In *Proceedings of the Human Language Technology Conference of the NAACL, Companion Volume: Short Papers*, pages 57–60, New York City, USA, June 2006. Association for Computational Linguistics.
- [24] Wenpeng Hu, Zhou Lin, Bing Liu, Chongyang Tao, Zhengwei Tao, Jinwen Ma, Dongyan Zhao, and Rui Yan. Overcoming catastrophic forgetting for continual learning via model adaptation. In *International Conference on Learning Representations*, 2018.
- [25] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*, pages 2261–2269. IEEE Computer Society, 2017.
- [26] Todd Huster and Emmanuel Ekwedike. Top: Backdoor detection in neural networks via transferability of perturbation. *arXiv preprint arXiv:2103.10274*, 2021.
- [27] Forrest N. Iandola, Matthew W. Moskewicz, Khalid Ashraf, Song Han, William J. Dally, and Kurt Keutzer. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and <1mb model size, 2016.
- [28] IARPA. Trojai competition. <https://pages.nist.gov/trojai/>.
- [29] Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Logan Engstrom, Brandon Tran, and Aleksander Madry. Adversarial examples are not bugs, they are features. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 125–136, 2019.
- [30] Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in neural information processing systems*, 31, 2018.

- [31] Matthew Jagielski, Alina Oprea, Battista Biggio, Chang Liu, Cristina Nita-Rotaru, and Bo Li. Manipulating machine learning: Poisoning attacks and countermeasures for regression learning. In 2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings, 21-23 May 2018, San Francisco, California, USA, pages 19–35. IEEE Computer Society, 2018.
- [32] Hengrui Jia, Christopher A Choquette-Choo, Varun Chandrasekaran, and Nicolas Papernot. Entangled watermarks as a defense against model extraction. In 30th USENIX Security Symposium (USENIX Security 21), pages 1937–1954, 2021.
- [33] Ronald Kemker, Marc McClure, Angelina Abitino, Tyler L. Hayes, and Christopher Kanan. Measuring catastrophic forgetting in neural networks. In Sheila A. McIlraith and Kilian Q. Weinberger, editors, Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018, pages 3390–3398. AAAI Press, 2018.
- [34] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. Proceedings of the national academy of sciences, 114(13):3521–3526, 2017.
- [35] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In International Conference on Machine Learning, pages 3519–3529. PMLR, 2019.
- [36] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.
- [37] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11):2278–2324, 1998.
- [38] Yige Li, Xixiang Lyu, Nodens Koren, Lingjuan Lyu, Bo Li, and Xingjun Ma. Neural attention distillation: Erasing backdoor triggers from deep neural networks. In 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021. OpenReview.net, 2021.
- [39] Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. Fine-pruning: Defending against backdooring attacks on deep neural networks. In Michael Bailey, Thorsten Holz, Manolis Stamatogiannakis, and Sotiris Ioannidis, editors, Research in Attacks, Intrusions, and Defenses - 21st International Symposium, RAID 2018, Heraklion, Crete, Greece, September 10-12, 2018, Proceedings, volume 11050 of Lecture Notes in Computer Science, pages 273–294. Springer, 2018.
- [40] Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. Fine-pruning: Defending against backdooring attacks on deep neural networks. In International Symposium on Research in Attacks, Intrusions, and Defenses, pages 273–294. Springer, 2018.
- [41] Yingqi Liu, Wen-Chuan Lee, Guan hong Tao, Shiqing Ma, Yousra Aafer, and Xiangyu Zhang. Abs: Scanning neural networks for backdoors by artificial brain stimulation. In Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, pages 1265–1282, 2019.
- [42] Yingqi Liu, Wen-Chuan Lee, Guan hong Tao, Shiqing Ma, Yousra Aafer, and Xiangyu Zhang. ABS: scanning neural networks for backdoors by artificial brain stimulation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019, pages 1265–1282. ACM, 2019.
- [43] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. CoRR, abs/1907.11692, 2019.
- [44] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. CoRR, abs/1907.11692, 2019.
- [45] Yunfei Liu, Xingjun Ma, James Bailey, and Feng Lu. Reflection backdoor: A natural backdoor attack on deep neural networks. In Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm, editors, Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part X, volume 12355 of Lecture Notes in Computer Science, pages 182–199. Springer, 2020.
- [46] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, pages 142–150, Portland, Oregon, USA, June 2011. Association for Computational Linguistics.
- [47] Cuong V Nguyen, Yingzhen Li, Thang D Bui, and Richard E Turner. Variational continual learning. arXiv preprint arXiv:1710.10628, 2017.
- [48] Jianmo Ni, Jiacheng Li, and Julian McAuley. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 188–197, 2019.
- [49] Ren Pang, Zheng Zhang, Xiangshan Gao, Zhaohan Xi, Shouling Ji, Peng Cheng, and Ting Wang. TROJANZOO: everything you ever wanted to know about neural backdoors (but were afraid to ask). CoRR, abs/2012.09302, 2020.
- [50] Xiangyu Qi, Tinghao Xie, Saeed Mahloujifar, and Prateek Mittal. Circumventing backdoor defenses that are based on latent separability. arXiv preprint arXiv:2205.13613, 2022.
- [51] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [52] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter. CoRR, abs/1910.01108, 2019.
- [53] Guangyu Shen, Yingqi Liu, Guan hong Tao, Shengwei An, Qiuling Xu, Siyuan Cheng, Shiqing Ma, and Xiangyu Zhang. Backdoor scanning for deep neural networks through k-arm optimization. arXiv preprint arXiv:2102.05123, 2021.
- [54] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In Yoshua Bengio and Yann LeCun, editors, 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings, 2015.
- [55] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel. Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. Neural Networks, (0):–, 2012.
- [56] Farhana Sultana, Abu Sufian, and Paramartha Dutta. Advancements in image classification using convolutional neural network. In 2018 Fourth International Conference on Research in Computational Intelligence and Communication Networks (ICRCICN), pages 122–129. IEEE, 2018.
- [57] Zhiqing Sun, Hongkun Yu, Xiaodan Song, Renjie Liu, Yiming Yang, and Denny Zhou. Mobilebert: a compact task-agnostic BERT for resource-limited devices. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel R. Tetreault, editors, Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020, pages 2158–2170. Association for Computational Linguistics, 2020.

- [58] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*, pages 2818–2826. IEEE Computer Society, 2016.
- [59] Di Tang, XiaoFeng Wang, Haixu Tang, and Kehuan Zhang. Demon in the variant: Statistical analysis of dnns for robust backdoor contamination detection. In Michael Bailey and Rachel Greenstadt, editors, *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, pages 1541–1558. USENIX Association, 2021.
- [60] Di Tang, XiaoFeng Wang, Haixu Tang, and Kehuan Zhang. Demon in the variant: Statistical analysis of dnns for robust backdoor contamination detection. In Michael Bailey and Rachel Greenstadt, editors, *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, pages 1541–1558. USENIX Association, 2021.
- [61] Ruixiang Tang, Mengnan Du, Ninghao Liu, Fan Yang, and Xia Hu. An embarrassingly simple approach for trojan attack in deep neural networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 218–228, 2020.
- [62] Ruixiang Tang, Mengnan Du, Ninghao Liu, Fan Yang, and Xia Hu. An embarrassingly simple approach for trojan attack in deep neural networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 218–228, 2020.
- [63] Erik F. Tjong Kim Sang and Fien De Meulder. Introduction to the conll-2003 shared task: Language-independent named entity recognition. In *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003 - Volume 4, CONLL '03*, page 142–147, USA, 2003. Association for Computational Linguistics.
- [64] Brandon Tran, Jerry Li, and Aleksander Madry. Spectral signatures in backdoor attacks. In Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 8011–8021, 2018.
- [65] Alexander Turner, Dimitris Tsipras, and Aleksander Madry. Label-consistent backdoor attacks. *CoRR*, abs/1912.02771, 2019.
- [66] Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y. Zhao. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*, pages 707–723. IEEE, 2019.
- [67] Tianyi Wang, Yating Zhang, Xiaozhong Liu, Changlong Sun, and Qiong Zhang. Masking orchestration: Multi-task pretraining for multi-role dialogue representation learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 9217–9224, 2020.
- [68] Ralph Weischedel and Ada Brunstein. Bbn pronoun coreference and entity type corpus. *Linguistic Data Consortium, Philadelphia*, 112, 2005.
- [69] Emily Wenger, Josephine Passananti, Arjun Nitin Bhagoji, Yuanshun Yao, Haitao Zheng, and Ben Y. Zhao. Backdoor attacks against deep learning systems in the physical world. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*, pages 6206–6215. Computer Vision Foundation / IEEE, 2021.
- [70] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In Qun Liu and David Schlangen, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, EMNLP 2020 - Demos, Online, November 16-20, 2020*, pages 38–45. Association for Computational Linguistics, 2020.
- [71] Baoyuan Wu, Hongrui Chen, Mingda Zhang, Zihao Zhu, Shaokui Wei, Danni Yuan, Chao Shen, and Hongyuan Zha. Backdoorbench: A comprehensive benchmark of backdoor learning. *arXiv preprint arXiv:2206.12654*, 2022.
- [72] Zhen Xiang, David J Miller, and George Kesidis. Detection of backdoors in trained classifiers without access to the training set. *IEEE Transactions on Neural Networks and Learning Systems*, 2020.
- [73] Ju Xu and Zhanxing Zhu. Reinforced continual learning. *arXiv preprint arXiv:1805.12369*, 2018.
- [74] Ke Xu, Yingjiu Li, Robert H. Deng, and Kai Chen. Deeprefiner: Multi-layer android malware detection system applying deep neural networks. In *2018 IEEE European Symposium on Security and Privacy, EuroS&P 2018, London, United Kingdom, April 24-26, 2018*, pages 473–487. IEEE, 2018.
- [75] Xiaojun Xu, Qi Wang, Huichen Li, Nikita Borisov, Carl A Gunter, and Bo Li. Detecting ai trojans using meta neural analysis. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 103–120. IEEE, 2021.
- [76] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. In Richard C. Wilson, Edwin R. Hancock, and William A. P. Smith, editors, *Proceedings of the British Machine Vision Conference 2016, BMVC 2016, York, UK, September 19-22, 2016*. BMVA Press, 2016.
- [77] Yi Zeng, Si Chen, Won Park, Z Morley Mao, Ming Jin, and Ruoxi Jia. Adversarial unlearning of backdoors via implicit hypergradient. *arXiv preprint arXiv:2110.03735*, 2021.
- [78] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*, pages 6848–6856. Computer Vision Foundation / IEEE Computer Society, 2018.
- [79] Fubang Zhao, Zhuoren Jiang, Yangyang Kang, Changlong Sun, and Xiaozhong Liu. Adjacency list oriented relational fact extraction via adaptive multi-task learning. *arXiv preprint arXiv:2106.01559*, 2021.
- [80] Xin Zhou, Yating Zhang, Xiaozhong Liu, Changlong Sun, and Luo Si. Legal intelligence for e-commerce: Multi-task learning by leveraging multiview dispute representation. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 315–324, 2019.
- [81] Liuwan Zhu, Rui Ning, Cong Wang, Chunsheng Xin, and Hongyi Wu. Gangsweep: Sweep out neural backdoors by gan. In *Proceedings of the 28th ACM International Conference on Multimedia*, pages 3173–3181, 2020.

Appendix

1. Theoretical proofs

In this section, we give the proof of the lemmas and theorems for the multi-class classification tasks. The same results also hold for binary classification tasks, for which the proofs are given in an online document ¹ (along with the detailed explanation about the independence between the residual and the target task) due to the space limit. Note that in Section 4, the results are illustrated using the binary classification tasks.

We consider the multi-class classification problem on L classes. We define L binary classifiers $\{f^{(j)}\}_{j \in \{1, 2, \dots, L\}}$,

1. <https://drive.google.com/file/d/1bPT0dbu0cVeBH7qRzAa76nEqyQ27Zh66/view?usp=sharing>

in which the j^{th} classifier $f^{(j)*}$ outputs the probability of a given input to be in the j^{th} class. For a dataset X with n samples in L classes, $X = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$, the label $y^{(i)}$ of the i^{th} input is a vector containing L values, i.e., $y^{(i)} = [y^{(i,1)}, y^{(i,2)}, \dots, y^{(i,L)}]$, and $y^{(i,1)}, y^{(i,2)}, \dots, y^{(i,L)} \in \{0, 1\}$. In this multi-class setting, we define the Catastrophe Forgetting (CF) as the following.

Definition of CF in a multi-class classifier: The CF from task τ_P to task τ_F w.r.t the dataset X_{τ_F} is:

$$\Delta^{\tau_P \rightarrow \tau_F}(X_{\tau_F}) = \sum_{x \in \mathcal{D}_{\tau_F}} (f_{\tau_F}^{(k)*}(x) - f_{\tau_P}^{(k)*}(x))^2 \quad (6)$$

where $k \in K = \{k | f_{\tau_P}^{(k)*}(x) = \max_{1 \leq j \leq L} f_{\tau_P}^{(j)*}(x)\}$.

For the sake of simplicity, we denote the symbol $\ominus$ as the operator: $f_{\tau_F}^*(x) \ominus f_{\tau_P}^*(x) = f_{\tau_F}^*(x) - f_{\tau_P}^*(x)$. Thus,

$$\Delta^{\tau_P \rightarrow \tau_F}(X_{\tau_F}) = \sum_{x \in \mathcal{D}_{\tau_F}} (f_{\tau_F}^*(x) \ominus f_{\tau_P}^*(x))^2 \quad (7)$$

This definition of CF for multi-class classifiers accounts for the change of confidence from $\tau_P \rightarrow \tau_F$, w.r.t the desirable class of an input sample for the task τ_P , that is, how much confidence is reduced for the desirable class in τ_P (equivalent to the confidence increased for the other classes) via the learning procedure $\tau_P \rightarrow \tau_F$.

Note that the residual is the only term that is different in the definition of the CF for multi-class classification tasks comparing with the definition of the CF for the binary classification tasks (see the online document). The residual term of the multi-class classification tasks can be written as:

$$\tilde{y}_{\tau_F} = y_{\tau_F} \ominus f_{\tau_P}^*(X_{\tau_F})$$

With the CF definition above, we can show that both the upper bound and the lower bound of CF is proportional to $\|\tilde{y}_{\tau_F}\|^2$ (Lemma 4.2), and when $\|\tilde{y}_{\tau_F}\|^2$ reach maximum, both the upper bound and the lower bound of the CF reach maximum (Theorem 4.3).

Lemma 4.1.5 Let $M \in \mathbb{R}^{n \times n}$ be a symmetric and non-singular matrix, and $v \in \mathbb{R}^n$ is a vector. Then $\lambda_{min}^2 \|v\|^2 \leq \|Mv\|_2^2 \leq \lambda_{max}^2 \|v\|^2$.

Lemma 4.2 (multi-class version) For any specific sample X , both the upper bound and the lower bound of CF from the source model $f_{\tau_P}^*$ to a competitive model trained on X_{τ_F} is proportional to the norm of residual: $\|\tilde{y}_{\tau_F}\|^2 = y_{\tau_F} - f_{\tau_P}^*(X_{\tau_F})$.

Proof. To show that CF is proportional to residual, we only need to show that for all elements in the residual, $\tilde{y}_{\tau_F}$ is greater than 0.

$$\Delta^{\tau_P \rightarrow \tau_F}(X) = \left\| \underbrace{\phi(X) \phi(X_{\tau_F})^\top}_A \underbrace{[\phi(X_{\tau_F}) \phi(X_{\tau_F})^\top + \lambda I]^{-1}}_B \tilde{y}_{\tau_F} \right\|_2^2 \quad (8)$$

, where A and B are two symmetric and non-singular kernel matrices.

Let $\lambda_{(A),min}^2$ and $\lambda_{(A),max}^2$ be the smallest and largest eigen-value of A , and $\lambda_{(B),min}^2$ and $\lambda_{(B),max}^2$ be the smallest

and the largest eigen-value of B . Applying Lemma 4.1.5, we have:

$$\lambda_{(A),min}^2 B \|\tilde{y}_{\tau_F}\|^2 \leq \Delta^{\tau_P \rightarrow \tau_F} \leq \lambda_{(A),max}^2 B \|\tilde{y}_{\tau_F}\|^2 \quad (9)$$

Then applying Lemma 4.1.5 again:

$$\lambda_{(A),min}^2 \lambda_{(B),min}^2 \|\tilde{y}_{\tau_F}\|^2 \leq \Delta^{\tau_P \rightarrow \tau_F} \leq \lambda_{(A),max}^2 \lambda_{(B),max}^2 \|\tilde{y}_{\tau_F}\|^2 \quad (10)$$

Therefore, both the upper bound and the lower bound of CF is proportional to the residual $\|\tilde{y}_{\tau_F}\|^2$. $\square$

Theorem 4.3 (multi-class version) Given a fixed input of a training dataset X_{τ_F} , the randomly assigned wrong label y_{τ_F} maximizes the norm of residual $\|\tilde{y}_{\tau_F}\|^2$, and thus maximizes both the upper bound and the lower bound of CF for any input sample X from the source model to the competitive model trained on the labeled dataset $\mathcal{D}_{\tau_F} = (X_{\tau_F}, y_{\tau_F})$.

Proof. Here, we prove how the randomly assigned wrong label y_{τ_F} maximizes the upper bound of CF. The proof for the lower bound is similar, and thus is not shown.

Lemma 4.2 shows that the upper bound of CF is proportional to $\|\tilde{y}_{\tau_F}\|^2$, i.e., $\sup \Delta^{\tau_P \rightarrow \tau_F} \propto \|\tilde{y}_{\tau_F}\|^2$. Thus the maximum of $\|\tilde{y}_{\tau_F}\|^2$ leads to the maximum of $\sup \Delta^{\tau_P \rightarrow \tau_F}$.

Next, we show the randomly assigned wrong labels y_{τ_F} can maximize $\|\tilde{y}_{\tau_F}\|^2$. Recall the residual term in the CF of multi-class classification models is defined as: $\tilde{y}_{\tau_F} = y_{\tau_F} \ominus f_{\tau_P}^*(X_{\tau_F})$. Let $(x_{\tau_F}^{(i)}, y_{\tau_F}^{(i)})$ be the i^{th} data point in the training dataset of the task τ_F . $\|\tilde{y}_{\tau_F}^{(i)}\|^2 = \|y_{\tau_F}^{(i)} - f_{\tau_P}^{(k)*}(x_{\tau_F}^{(i)})\|^2$ is the residual norm of $x_{\tau_F}^{(i)}$ from task τ_P to task τ_F . Specifically, the first term in the residual norm is $y_{\tau_F}^{(i,k)}$, the k^{th} label of $x_{\tau_F}^{(i)}$, and $y_{\tau_F}^{(i,k)} \in \{0, 1\}$. The second term is $f_{\tau_P}^{(k)*}(x_{\tau_F}^{(i)})$, the predicted outcome for the k^{th} class by the model for the task τ_P on $x_{\tau_F}^{(i)}$, and $f_{\tau_P}^{(k)*}(x_{\tau_F}^{(i)}) \in [0, 1]$. Since in the task τ_F , the label of the i^{th} input is the randomly wrong label, i.e., $y_{\tau_F}^{(i,k)} = 0 \neq 1 = y_{\tau_P}^{(i,k)}$, and the i^{th} input is the same as the i^{th} input of the task τ_P , i.e., $x_{\tau_F}^{(i)} = x_{\tau_P}^{(i)}$, we have $f_{\tau_P}^{(k)*}(x_{\tau_F}^{(i)}) = f_{\tau_P}^{(k)*}(x_{\tau_P}^{(i)}) = y_{\tau_P}^{(i,k)} = 1$ for a well trained classifier $f_{\tau_P}^*$. Thus, $\|y_{\tau_F}^{(i,k)} - f_{\tau_P}^{(k)*}(x_{\tau_F}^{(i)})\|^2 = 1$ reaches the maximum, i.e., the theorem is proved. $\square$

2. Dataset

TrojAI competition datasets. TrojAI is a competition founded by IARPA for backdoor detection. This competition includes several rounds that are designed for different image processing and NLP tasks. In each round, a set of training datasets, testing datasets and holdout datasets are published, which contain benign and infected models with various structures. In our research, we evaluated SEAM on all three datasets released for *Round 3-7*. Specifically, *Round 3-4* are designed for backdoor detection on image classification tasks and *Round 5-7* for backdoor detection on NLP tasks.

The dataset of *Round 3* has totally 1584 models with 22 different model structures in 7 types of architectures: ResNet {18, 34, 50, 101, 152} [21], wide-ResNet {50,

TABLE 8: Datasets statistics. The training samples and testing samples of models in *Round 3-7* of TrojaAI competition are different one by one, and the details are stored in configuration file associated with each model. Here, we only list the approximate value for reference.

	MNIST	GTSRB	CIFAR10	Round 3	Round 4	Round 5	Round 6	Round 7
Models (#)	12	12	12	1584	1584	2664	3114	960
Training samples (#)	60,000	39,209	50,000	~40,000	~40,000	~100,000	~40,000	~40,000
Testing samples (#)	10,000	12,630	10,000	~4,000	~4,000	~10,000	~4,000	~4,000

101} [76], DenseNet {121, 161, 169, 201} [25], Inception {v1, v3} [58], SqueezeNet {v1.0, v1.1, v2} [27], ShuffleNet {1.0, 1.5, 2.0} [78], and Vgg {11bn, 13bn, 16bn} [54]. Backdoors injected into these models can be universal (mapping all labels to a target one) or specific (mapping a specific source label to a target), and multiple backdoors can present in one model. The triggers of the backdoors include pixel patterns (e.g., polygons with solid color) and Instagram filters that are applied to images to change their styles (e.g., Gotham Filter).

The dataset of *Round 4* contains 1584 models with 16 structures. Triggers in this round are more subtle than those in *Round 3*. They could be spatially dependent, only taking effect when they appear at specific locations relative to the foreground objects in images, and spectral dependent, requiring a right combination of colors in order to cause misclassification.

The dataset of *Round 5* includes 2664 sentiment classification models trained on the IMDB movie review dataset [46] and the Amazon review dataset [48] in 3 popular NLP model architectures: BERT [10], GPT-2 [51] and DistilBERT [52]. A trigger in this round can be a character (e.g., “@”), a word (e.g., “cromulent”) or a phrase (e.g., “I watched an 3D movie”). The dataset of *Round 6* share the same settings except that its dataset for training detector is very small with only 48 models. Thus, together with the training, testing and holdout datasets, we used 3114 models of *Round 6* in our experiments.

The dataset of *Round 7* carries 960 models for Named-Entity Recognition (NER). These models trained on BBN [68], CoNLL-2003 [63] and OntoNotes [23] NER datasets, and are in 4 different model architectures: BERT [10], DistilBERT [52], RoBERTa [44] and MobileBERT [57]. Different from binary classification in *Round 6*, the NER task of *Round 7* involves multiple classes, which assign each word into one of the several categories. It uses the trigger settings of *Round 6*.

MNIST, GTSRB, CIFAR10. These three datasets are constructed for image classification tasks: MNIST is for recognizing handwritten digits, GTSRB is for detecting traffic signs and CIFAR10 is for classifying general images. On these datasets, we trained 3 models for each of the 4 model architectures that includes ShuffleNetx1.0, Vgg16, ResNet18 and ResNet101. Totally 12 models trained on each dataset were used for backdoor injections (e.g., TrojanNet [61]) to get backdoor infected models.

3. Additional Experiments

Backdoor-revival experiments.

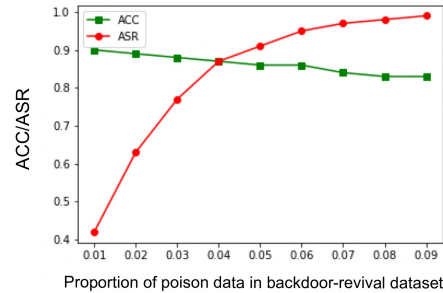


Figure 7: ASR and ACC changes when the adversary fine-tunes an unlearned model on a backdoor-revival dataset with different poison proportions.

We ran SEAM on the backdoored models in four mainstream architectures (ShuffleNetx1.0, Vgg16, ResNet18, and ResNet101), and further fine-tuned the unlearned models on a poisoned dataset (referred to as the “backdoor-revival” dataset). Specifically, we constructed the backdoor-revival dataset by randomly sampling 10,000 benign images from CIFAR10 used to train these models and replacing a portion (1% - 9%) of them with trigger-carrying instances. Our experimental results are presented in Fig. 7. As we can see from the figure, the backdoors within these models can be revived ($ASR \geq 89\%$) through fine-tuning them on the backdoor-revival dataset when the portion of trigger-carrying inputs grows above 4%.

CKA experiments of EW on CIFAR100. On CIFAR100, we ran SEAM on a EW infected ResNet-18 model with a clean dataset with the size of 0.1% of the training dataset for forgetting and a clean dataset with the size of 10% of the training dataset for recovery, and calculated the CKA for each layer. The results are demonstrated on Fig. 8. We observe that, even the backdoor has been entangled with the primary task by the EW, there is still chance to depress this backdoor without harming the primary task performance of the target model through SEAM that preserves many features of the original model on shallow layers.

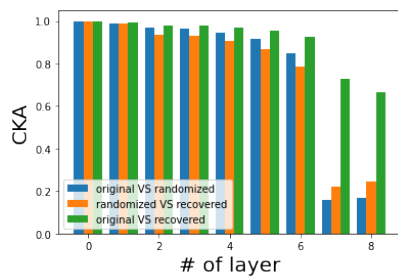


Figure 8: CKA on each layer of ResNet-18. Layer 0 is the first layer taking the image as its input and layer 9 is the last second layer.